Troubleshooting Journal

Data and analytics work at Nexora: pandas, PostgreSQL, Metabase and Excel

Volume 1 · October to November 2026

Days logged	Entries	Errors	Wrong results	How-tos
9	117	29	32	56

Contents

How this journal works	Structure, the log card fields and the daily routine
Index	Every entry on one page: search by tool, type or tag
Patterns	Lessons that came up more than once
Day pages	29 October 2026: 13 entries · 30 October 2026: 21 entries · 31 October 2026: 12 entries · 2 November 2026: 10 entries · 4 November 2026 (covers 3 to 4 Nov): 11 entries · 9 November 2026: 15 entries · 10 November 2026: 11 entries · 11 November 2026: 15 entries · 12 November 2026: 9 entries
Templates	A blank day page and log card to copy for each new day

How this journal works

Every problem you solve becomes one log card, filed under the day it happened. The Index lets you find any card by tool, type or tag, and the Patterns page collects the lessons that keep coming back.

The four parts

Part	What it holds	Update it
Index	One line per card: ID, tool, type, title, tags	After every new card
Patterns	Lessons seen on two or more cards, with what to do	Weekly review
Day pages	Date, focus, outcome, then the day's log cards	During and at the end of each day
Templates	A blank day page and log card	Copy one for each new day

The log card

Field	What to write
ID	Date plus a running number: `20261029-05` is the 5th card on 29 Oct 2026. Sorts by date automatically.
Tool	pandas, SQL, Excel, Python, Metabase, Bash.
Type	**Error**: a message appeared. **Wrong result**: no error, but the output was wrong. **How-to**: a new task you worked out.
Tags	2 to 4 search words: the function or concept, e.g. `ROW_NUMBER`, merge, dates.
Problem	Exactly what you saw. Paste the error text word for word, so you can search for it later.
Cause	Why it happened, in one or two sentences.
Fix	The code that worked, trimmed to the lines that matter.
Why it works	The two to four details you would forget in a month.
Lesson	One line you could apply to a different problem.

The routine

1. **As soon as a problem is solved**, write its card. Five minutes now saves an hour of re-asking later.
2. **At the end of the day**, fill the day's Focus and Outcome lines and add the cards to the Index.
3. **Once a week**, scan the Index. Any lesson that appears on two cards goes on the Patterns page.
4. **Once a month**, start a new volume. Carry the Index and Patterns forward.
5. **Before asking for help**, search the Index for the error text or a tag. The answer may already be here.

Keeping it searchable. Paste error messages exactly, including quotes and capitals. Searching this PDF (Ctrl+F) for `does not exist` or `has no attribute` finds every related card.

Index

One line per log card, oldest first. Types: **Error** = a message appeared, **Wrong result** = wrong output with no error, **How-to** = a new task.

ID	Tool	Type	Title	Tags
20261029-01	pandas	How-to	Name the dialer output file with the Start Time date range	dates, f-string, files
20261029-02	pandas	Error	Splitting Caller Ids: [1] vs .str[1], and "'DataFrame' object has no attribute 'str'"	strings, duplicate columns, .str
20261029-03	pandas	How-to	Extract customer and agent_id from Contact Info with regex	regex, str.extract, nullable ints
20261029-04	pandas	How-to	Combine Call Date and a 12-hour Call Time into one datetime	dates, parsing, 12-hour clock
20261029-05	SQL	Wrong result	Leads whose last 5 calls are all one disposition	ROW_NUMBER, HAVING, BOOL_AND
20261029-06	SQL	Wrong result	Raw verification query showed non-Zero dispositions	window functions, WINDOW, verification
20261029-07	SQL	How-to	Latest disposition is Zero, plus the distribution of the last 5	FILTER, STRING_AGG, two-level aggregation
20261029-08	SQL	How-to	Unreachable list, name normalisation, and last-10 pivot with call times	regexp_replace, VALUES, pivot
20261029-09	SQL	Error	column ol.client_id does not exist	Metabase, column names
20261029-10	SQL	Error	syntax error at or near "=" (Position 5128)	JOIN syntax, aliases
20261029-11	SQL	Wrong result	Leads repeated after joining customer_status_history	one-to-many join, DISTINCT ON, ARRAY_AGG
20261029-12	pandas	Error	TypeError: Can only merge Series or DataFrame objects, a <class 'str'> was passed	merge, latest row per key
20261029-13	Excel	How-to	Tag Active-but-unreachable clients as automatic traders	IF, COUNTIF, confidence levels
20261030-01	pandas	How-to	Select ledger rows for yesterday	dates, half-open range, sort_values
20261030-02	Polars	Wrong result	The combined ledger seemed to stop at the 27th	ledger_loader, diagnostics, Excel-saved CSV
20261030-03	Excel	Wrong result	SUMIFS returned 0 for negative balance protection	SUMIFS, $ references, COUNTIF
20261030-04	Excel	How-to	Extract the number from BR575346: ****	REGEXEXTRACT, MID, FIND
20261030-05	Excel	Error	The formula shows as text instead of calculating	cell format, Show Formulas
20261030-06	Excel	Wrong result	UNIQUE(FILTER(..., OR(...))) doesn't filter	FILTER, OR logic, arrays
20261030-07	pandas	Error	KeyError: 'ClientID' in groupby	variable names, columns
20261030-08	pandas	How-to	Fill Zynixx Balance and Last Trade with .map()	map, lookup, set_index

ID	Tool	Type	Title	Tags
20261030-09	pandas	Wrong result	Latest balance taken from a random row	drop_duplicates, latest row per key
20261030-10	pandas	Error	KeyError: Index([Ellipsis], dtype='object')	placeholder code
20261030-11	pandas	How-to	Add a result sheet to the original workbook	ExcelWriter, openpyxl
20261030-12	pandas	Error	Combining every file in a folder	os.listdir, concat, built-in names
20261030-13	pandas	Error	DtypeWarning: Columns (21: expiryDate) have mixed types	dtype, read_csv, mixed types
20261030-14	pandas	Error	MemoryError: Unable to allocate 487 MiB ... shape (1, 63807297)	memory, usecols, concat
20261030-15	Polars	Error	'LazyFrame' object has no attribute 'to_pandas'	LazyFrame, collect
20261030-16	Polars	Error	Kernel crashed on lf.collect()	memory, streaming, sink
20261030-17	Polars	How-to	Append the ark folder (CSV and XLSX) to the transactions	read_excel, diagonal_relaxed, source_file
20261030-18	Polars	How-to	transactions_loader.py: a module that can't crash the kernel	module, select before collect
20261030-19	pandas	Wrong result	Aligning ark columns gave four amount columns	rename, normalised names, clash check
20261030-20	pandas	How-to	Match the main file's columns and types exactly (minus id)	reindex, Int64, dtypes
20261030-21	pandas	Wrong result	Times show as 59:48.4 (date and hour lost)	Excel, CSV, data loss
20261031-01	Excel	Error	FILTER rejected: 'BILL' in single quotes	quotes, FILTER, UNIQUE
20261031-02	pandas	How-to	Organised, faster Stringee CDR combine script	pathlib, calamine, xlsxwriter, # %% cells
20261031-03	pandas	How-to	Read IDs as text; convert dates and durations after	dtype, to_timedelta, [h]:mm:ss
20261031-04	pandas	Wrong result	Old dialer notebook: Uno start times lost and durations zeroed	regression test, Uno, timedelta
20261031-05	pandas	How-to	Call gaps with groupby().shift() instead of a loop	groupby, shift, vectorise
20261031-06	pandas	Error	Grouper for 'Date' not 1-dimensional	pivot_table, keys
20261031-07	pandas	How-to	Multi-day reports with checks that fail loudly	concat, drop_duplicates, consistency check
20261031-08	pandas	Wrong result	Report showed only the latest date: wrong input folder	inputs, folders, debugging order
20261031-09	pandas	How-to	Parse 10/28/2026 09:34:25 PM safely	to_datetime, format, month-first
20261031-10	pandas	How-to	Use the newest team list in a folder	pathlib, st_mtime
20261031-11	Python	Error	KeyError on os.environ: token used as the variable name	environment variables, setx, secrets

ID	Tool	Type	Title	Tags
20261031-12	pandas	Wrong result	Over 31 trading days in a month	nunique, normalize, assert
20261102-01	SQL	How-to	Lead source for depositing customers	DISTINCT ON, self-join, COALESCE
20261102-02	pandas	How-to	Flag which customers traded in October	isin, np.where, Period
20261102-03	pandas	How-to	Total and traded clients per Manager and agent	groupby, nunique, where, subtotals
20261102-04	SQL	Wrong result	Calls after conversion would repeat per lead	fan-out, MIN, FILTER, STRING_AGG
20261102-05	SQL	How-to	Start from the pasted ID list so no ID disappears	UNNEST, ARRAY, LEFT JOIN, check_status
20261102-06	SQL	How-to	Join a Metabase saved question safely	{{#3268}}, DISTINCT ON, team list
20261102-07	SQL	How-to	Keep only the Sales team's calls	JOIN vs LEFT JOIN, filter on joined table
20261102-08	SQL	How-to	30-day counts beside the day buckets	FILTER, INTERVAL, cross-check
20261102-09	Process	Wrong result	A 30-day request rebuilt three times	requirements, output columns
20261102-10	Metabase	How-to	Admin IDs that share an IP	Summarize, distinct count, STRING_AGG, private IPs
20261104-01	pandas	How-to	When a lambda is fine, and when it's 900x slower	apply, vectorise, benchmark
20261104-02	pandas	How-to	Deposits and withdrawals net of cancellations	where, groupby, ledger types
20261104-03	pandas	Error	np.where got one argument: a bracket closed too late	np.where, &, brackets
20261104-04	pandas	Wrong result	Latest balance taken from a random row (again)	sort_values, drop_duplicates, repeated
20261104-05	pandas	Wrong result	Campaign flag: 'deposited on' vs 'first deposit was'	first row per group, requirements
20261104-06	pandas	How-to	Trading days and their spread per client	nunique, join per group, to_period
20261104-07	pandas	How-to	Excel export in the MIS design	openpyxl, ExcelWriter, number_format
20261104-08	pandas	Wrong result	NBP always 0: rows filtered away one step early	filter order, isin
20261104-09	Python	Error	TypeError: insert expected 2 arguments, got 1	sys.path, imports, PYTHONPATH
20261104-10	Python	How-to	Writing functions in three levels	def, parameters, defaults, None, raise
20261104-11	pandas	How-to	Prove a rewrite didn't change the numbers	assert_frame_equal, testing
20261109-01	SQL	How-to	Call list: traded in the last 3 days, not called for 2	INTERVAL, NULL, OR brackets

ID	Tool	Type	Title	Tags
20261109-02	SQL	Error	interval 3 days fails; CURRENT_DATE AT TIME ZONE skews the gaps	INTERVAL, AT TIME ZONE, ::date, CTE
20261109-03	SQL	Error	ROW_NUMBER written wrong, and every status row joined	ROW_NUMBER, OVER, one-to-many
20261109-04	SQL	Wrong result	Latest status picked a row with no date: DESC puts NULLs first	ORDER BY, NULLS LAST, tie-breaker
20261109-05	SQL	Wrong result	Lost Lead shown as Dormant: several rows tie on the sort column (open)	ties, LIMIT 1, business rule
20261109-06	SQL	How-to	Team alignment from the lead's current owner, not the last caller	LEFT JOIN, models, name join
20261109-07	SQL	How-to	Remove clients who transacted today (Indian time)	::date, AT TIME ZONE, calendar day
20261109-08	Metabase	Error	syntax error at or near "$1": IN without brackets	variables, IN, [[ ]]
20261109-09	Metabase	Error	Multiple ResultSets were returned by the query	semicolon, statements
20261109-10	SQL	How-to	A Total row with ROLLUP, at the bottom or the top	ROLLUP, GROUPING, totals
20261109-11	SQL	How-to	DISTINCT ON and ROW_NUMBER give the same latest row	DISTINCT ON, ROW_NUMBER, IS DISTINCT FROM
20261109-12	Django	How-to	A sign-up page: built-in form, extra fields, admin approval	UserCreationForm, clean_email, is_active
20261109-13	Django	Wrong result	Every form error showed twice	templates, non_field_errors, as_p
20261109-14	Django	How-to	An unapproved user only sees 'incorrect username or password'	is_active, AuthenticationForm
20261109-15	Django	How-to	A DataFrame as a sortable, paged HTML table (Ledger explorer)	to_dict('records'), Paginator, request.GET
20261110-01	pandas	How-to	Every child in exactly one group: np.select with a priority	np.select, named agg, is_demo as text
20261110-02	pandas	Wrong result	isin(is_demo) compared client IDs with True and False	isin, boolean mask
20261110-03	pandas	Error	SettingWithCopyWarning when adding a Month column to a filtered table	SettingWithCopyWarning, Period, date filter
20261110-04	pandas	How-to	Broker program report: yes/no flags per child, summed per broker	flags, isin, groupby sum
20261110-05	pandas	How-to	Program targets: required active, new clients still to add, minimum trade value	clip, np.select, settings
20261110-06	pandas	Wrong result	A broker with no active clients was eligible without trading	edge case, floor
20261110-07	pandas	Wrong result	max() set the wrong target for a broker just below the minimum	np.where, either/or rule
20261110-08	pandas	How-to	Trade value as one column per month	to_period, pivot_table, reindex

ID	Tool	Type	Title	Tags
20261110-09	pandas	Wrong result	Lookup merges could duplicate brokers; is_broker listed twice	merge, validate, duplicate columns
20261110-10	pandas	How-to	What 'Active within 30 days' measures	rolling window, as_of
20261110-11	pandas	How-to	Active previous month, and where each new line goes	Period arithmetic, dependency order
20261111-01	Excel	How-to	LET: work a value out once, show a message when it's below 6	LET, IF, text joining
20261111-02	Excel	Wrong result	A message in a number column turned the target into #VALUE!	#VALUE!, mixed column, IFERROR
20261111-03	Excel	Wrong result	The plan's rules and the spreadsheet's formulas said different things	rule definitions, INT, demo children
20261111-04	Excel	How-to	Pull Client_Id out of JSON text in a cell	TEXTAFTER, TEXTBEFORE, REGEXEXTRACT, Flash Fill
20261111-05	pandas	Wrong result	113,534 repeated ledger rows in every month (all-time deposits too high?)	duplicates, SLNo., ledger overlap
20261111-06	pandas	Wrong result	A check compared two different sets of rows	diagnostics, like with like
20261111-07	pandas	How-to	All-time deposits for every client in the leads file	set of IDs, groupby, map
20261111-08	pandas	Wrong result	Filtering leads by lead date missed conversions of earlier leads	cohort vs activity, event date
20261111-09	pandas	Error	KeyError: 'converted_in_month' not found in columns (astype)	stale cell, astype
20261111-10	pandas	Wrong result	One source under three labels: Meta-IMR, (META-IMR), Meta 1yt IMR	labels, replace, value_counts
20261111-11	pandas	How-to	Mixed date formats: format='ISO8601' instead of errors='coerce'	to_datetime, ISO8601
20261111-12	pandas	How-to	The same report for any month: lead_summary(month)	function, MonthBegin, pd.concat dict
20261111-13	Python	Error	PermissionError: [Errno 13] Permission denied writing the Excel file	PermissionError, open file, try/except
20261111-14	Python	How-to	Export: one column list drives the bands, month-to-date title, a sheet per month	openpyxl, COLUMNS spec, ExcelWriter
20261111-15	Python	Error	TypeError: unexpected keyword argument 'temperature' (code from an older tutorial)	anthropic package, temperature, prefill
20261112-01	SQL	Wrong result	INTERVAL '3 days' is 72 hours, not three calendar days	AT TIME ZONE, date difference, CASE
20261112-02	pandas	Error	TypeError: Boolean array expected for the condition, not object	where, brackets, KeyError
20261112-03	Python	Error	ModuleNotFoundError: No module named 'ledger_loader' after a kernel restart	sys.path, restart, first cell
20261112-04	Python	Error	A sys.path entry missing its drive letter	Windows paths, os.path.isdir
20261112-05	Excel	How-to	Clients, Leads and Check sheets behind the lead report	COUNTIFS, SUMIFS, INDEX/MATCH

ID	Tool	Type	Title	Tags
20261112-06	Python	**Error**	ImportError: cannot import name 'export_lead_workbook'	old file, Downloads copy, reload
20261112-07	pandas	**How-to**	Why the clients table keeps leads with no converted_at	NaT, ledger vs CRM
20261112-08	NumPy	**How-to**	A language model by counting: bigram and trigram	tokens, probabilities, loss
20261112-09	NumPy	**Wrong result**	+1 smoothing drowned a tiny dataset	smoothing, data size

Patterns

Lessons that showed up on more than one card. Read this page first when something looks wrong but no error appears.

Pattern	Seen in	What to do
Latest row per key: sort by the date, fully	20261030-01 20261030-09 20261104-04 20261109-04 20261109-05	pandas: sort_values(date, ascending=False) right before drop_duplicates(keep='first'). SQL: ORDER BY date DESC NULLS LAST plus a unique tie-breaker in ROW_NUMBER / DISTINCT ON.
Check the inputs before the code	20261030-02 20261031-08 20261104-08 20261110-02 20261112-04	Print which files and rows are used (value_counts after each filter) before changing any logic.
Repeated rows mean a one-to-many join or duplicate keys	20261029-11 20261029-12 20261030-21 20261102-01 20261102-04 20261102-06 20261109-03 20261109-06 20261110-09 20261111-05	Collapse the many side to one row per key first. Check COUNT(*) = COUNT(DISTINCT key), validate='many_to_one', or duplicated(subset=key).
Agree the output before building	20261102-09 20261104-05 20261109-05 20261109-06 20261110-10 20261111-03 20261111-08	Name the columns to add, what to keep, and the exact rule (first / any / only; owner or caller; which status wins a tie).
Check what is really stored before filtering	20261029-05 20261029-08 20261029-09 20261030-03 20261030-07 20261102-07 20261109-05 20261109-07 20261111-05 20261111-10	Look at real values and column names: df.columns, SELECT DISTINCT, value_counts, COUNTIF with wildcards.
One label per row when groups must add up	20261110-01 20261110-05 20261111-08	np.select([...], [...], default=...) gives each row exactly one label, decided in a stated order; the groups then always sum to the total.
Try the edge cases of every business rule	20261110-06 20261110-07 20261111-03 20261112-09	Before trusting a rule, run a zero row (no activity), a just-below row and a just-above row through it.
Name each setting and window once	20261110-05 20261110-08 20261111-12	Program numbers and date windows as named settings at the top (REPORT_MONTH, TRADE_MONTHS, MIN_TRADE_VALUE); never reuse one start/end for two meanings.

Pattern	Seen in	What to do
Every number counts by one stated date	20261110-10 20261111-08 20261111-10 20261112-01 20261112-07	Lead date, event date, rolling 30 days: name the date each column counts by, keep views apart, and print the window in every run and in the report's notes.
Decide what a blank means	20261109-01 20261109-04 20261112-07	For every filter or sort on a column that can be NULL: include or exclude (OR col IS NULL), and place them deliberately (NULLS FIRST / NULLS LAST).
Same type, same clock before date maths	20261109-02 20261109-07 20261110-10 20261112-01	Convert both sides to dates in one zone, (ts AT TIME ZONE 'Asia/Kolkata')::date, before subtracting; say whether a window is rolling hours or calendar days.
Start from the thing every row must represent	20261102-05	Every ID must appear? Start from the IDs and LEFT JOIN the rest; label why a value is zero.
Duplicate or double-used names break things	20261029-02 20261030-19 20261031-06 20261109-02 20261110-09 20261111-10	Two columns with one name, or one column as both pivot key and value. Check names after every rename.
Names, brackets and arguments	20261029-01 20261030-12 20261031-11 20261104-03 20261104-09 20261109-02 20261109-03 20261109-08 20261111-15 20261112-02	Built-in names, a token's value used as its name, a bracket closing late or missing, a missing argument, an unquoted literal.
Metabase: one statement, variables where values go	20261109-03 20261109-08 20261109-09	IN ({{var}}) with brackets, [[AND ...]] for optional filters, no trailing semicolon, test queries in their own question.
Verify against raw rows or an old version	20261029-06 20261029-07 20261029-13 20261030-02 20261031-04 20261104-11 20261109-11 20261110-04 20261111-06 20261112-05	Compare counts with raw data; run old and new code on the same input; assert_frame_equal; a query that counts differences.
Build checks that fail loudly	20261031-07 20261031-09 20261031-12 20261102-05 20261102-08 20261112-05	assert impossible values, raise on unreadable dates, cross-check two outputs, label every zero.
Prefer whole-column code to row loops	20261031-05 20261104-01 20261104-02 20261110-04	Column operators, np.where/select, groupby built-ins first; apply(lambda) only when nothing else fits.

Pattern	Seen in	What to do
Load only what you need	20261030-14 20261030-15 20261030-16 20261030-18	Estimate rows x columns x 8 bytes. Select columns and filter rows before collecting; sink big outputs to disk.
Text that looks like a date or number isn't one	20261029-04 20261029-12 20261030-13 20261030-20 20261031-03 20261031-09 20261111-02 20261111-11	Read IDs as text; give date formats explicitly; convert with to_datetime / to_timedelta / Int64.
Excel changes data when it saves	20261030-02 20261030-05 20261030-21	Never save a CSV from Excel. Long IDs become 2.61029E+17 and times become 59:48.4 for good.
Trust only output from a fresh run	20261030-07 20261030-16 20261031-12 20261110-09 20261110-11 20261111-09 20261111-13 20261112-03 20261112-06	Restart and Run All before trusting a number; close output files in Excel before writing them.
Put setup in the first cell	20261112-03 20261112-04 20261112-06	Imports, sys.path and helpers live in cell 1. After a restart or a replaced file, run from the top, and check that folders and files exist before importing from them.

SQL run order (the reason behind several patterns)

```
FROM / JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT → DISTINCT → ORDER BY → LIMIT
```

Thursday, 29 October 2026

Focus	Dialer file clean-up in pandas, the not-connected leads report in PostgreSQL (Metabase), and tagging Active-but-unreachable clients as automatic traders in Excel.
Outcome	13 entries: 4 errors, 3 wrong results and 6 how-tos. Final report delivered as Not_Connected_Report_v3.xlsx with 205 clients tagged (High 122, Medium 34, Low 49).
Cards	`20261029-01` to `20261029-13`

20261029-01 Name the dialer output file with the Start Time date range

pandas · **How-to** · dates, f-string, files

PROBLEM

The combined dialer file needed a name showing the first and last `Start Time` dates.

CAUSE

The dates were stored in variables named `min` and `max`, which replaces Python's built-in functions. Saving the output into the input folder would also break the "exactly 2 .xlsx files" check on the next run.

FIX

```
concat_df['Start Time'] = pd.to_datetime(concat_df['Start Time'], format='%Y-%m-%d %H:%M:%S.%f')
min_date = concat_df['Start Time'].min()
max_date = concat_df['Start Time'].max()

if min_date.date() == max_date.date():
    output_name = f"st_daily_{min_date:%Y-%m-%d}.xlsx"
else:
    output_name = f"st_daily_{min_date:%Y-%m-%d}_to_{max_date:%Y-%m-%d}.xlsx"

output_dir = r'C:\Users\Administrator\Documents\Dialer\st_output'
os.makedirs(output_dir, exist_ok=True)
concat_df.to_excel(os.path.join(output_dir, output_name), index=False)
```

WHY IT WORKS

- Inside an f-string, the text after the colon is a date format: `{min_date:%Y-%m-%d}` gives `2026-10-29`.
- `.date()` drops the time, so the `if` compares calendar days only.
- `os.makedirs(..., exist_ok=True)` creates the output folder only if it is missing.
- Windows file names can't contain colons, so use `%Y%m%d_%H%M` if you need times in the name.

> **Lesson:** Never name a variable after a built-in (`min`, `max`, `list`, `sum`). Keep outputs out of input folders.

20261029-02 Splitting Caller Ids: [1] vs .str[1], and "'DataFrame' object has no attribute 'str'"

pandas · **Error** · strings, duplicate columns, .str

PROBLEM

`uno_copy['Caller Ids'].str.split('|')[1]` gave the wrong result. The next attempt failed with `AttributeError: 'DataFrame' object has no attribute 'str'`.

CAUSE

`[1]` picks row label 1 of the whole column, not item 1 of each list. The AttributeError happened because two columns were named `Caller Ids`, so selecting the name returned a DataFrame, which has no `.str`.

FIX

```
# Find duplicated column names
print(uno_copy.columns[uno_copy.columns.duplicated()])

# Keep the first copy of each name (only if the copies are identical)
uno_copy = uno_copy.loc[:, ~uno_copy.columns.duplicated()]

# '917557160434 | 919810007919' -> '919810007919'
uno_copy['Caller Ids'] = uno_copy['Caller Ids'].str.split('|').str[1].str.strip()
```

WHY IT WORKS

- `.str` applies a method to every value. `.str[1]` takes item 1 from each row's list.
- `columns.duplicated()` is True for each repeat after the first. `~` flips it, and `.loc[:, mask]` keeps the True columns.
- If the duplicate columns hold different data, rename them (`Caller Ids_1`) instead of dropping.
- Rows without `|` become NaN. Add `.fillna(uno_copy['Caller Ids'])` to keep the original value there.

Lesson: If selecting one column name returns a DataFrame, there are duplicate column names.

20261029-03 Extract customer and agent_id from Contact Info with regex

pandas · **How-to** · regex, str.extract, nullable ints

PROBLEM

`Contact Info` holds text like `userId : 30628867 | callFirst : agent | customer : 919810007919 | agent_id : 8943 | agentType : agent_id`. Only `customer` and `agent_id` were needed.

CAUSE

Splitting on `|` depends on field order. A regex per field finds the value wherever it sits.

FIX

```
uno_copy['customer'] = uno_copy['Contact Info'].str.extract(r'customer\s*:\s*(\d+)', expand=False)
uno_copy['agent_id'] = uno_copy['Contact Info'].str.extract(r'\bagent_id\s*:\s*(\d+)', expand=False)

uno_copy['customer'] = uno_copy['customer'].str[-10:]          # drop the 91 prefix
uno_copy['agent_id'] = pd.to_numeric(uno_copy['agent_id'], errors='coerce').astype('Int64')
```

WHY IT WORKS

- `\s*` = any number of spaces, `\d+` = one or more digits, `( )` = the part returned.
- `\b` (word boundary) stops `sub_agent_id` from matching. `agentType : agent_id` doesn't match because no digits follow.
- `expand=False` returns one column. Rows without a match get NaN, not an error.
- `Int64` (capital I) allows blanks; plain `int64` would turn the column into floats like `8943.0`.

Lesson: One regex per field, so the order of fields doesn't matter.

20261029-04 Combine Call Date and a 12-hour Call Time into one datetime

pandas · **How-to** · dates, parsing, 12-hour clock

PROBLEM

`Call Date` (2026-10-28) and `Call Time` (11:58:14 pm) were separate columns.

CAUSE

The time uses a 12-hour clock with am/pm, so it needs `%I` and `%p`, not `%H`.

FIX

```
uno_copy['Call Datetime'] = pd.to_datetime(
    pd.to_datetime(uno_copy['Call Date']).dt.strftime('%Y-%m-%d')
    + ' '
    + uno_copy['Call Time'].astype(str).str.strip().str.upper(),
    format='%Y-%m-%d %I:%M:%S %p'
)
# '2026-10-28' + ' ' + '11:58:14 PM' -> 2026-10-28 23:58:14
```

WHY IT WORKS

- The inner `pd.to_datetime` makes it work whether `Call Date` is text or a datetime.
- `.dt.strftime` turns each date into text, and `+` joins text columns row by row.
- `.str.upper()` turns `pm` into `PM` so it matches `%p`.
- To find bad rows, add `errors='coerce'` and look at rows where the result `.isna()`.

Lesson: %H = 24-hour clock. %I + %p = 12-hour clock with AM/PM.

20261029-05 Leads whose last 5 calls are all one disposition

SQL · **Wrong result** · ROW_NUMBER, HAVING, BOOL_AND

PROBLEM

The original query used `DISTINCT ON` to get only the latest disposition. Filtering for `'0'` found nothing, and the old filter on Call Later / Interested / Prospect would always return 0 rows.

CAUSE

`DISTINCT ON` keeps one row per lead, so it can't look at 5 calls. The stored name is `'Zero'`, not `'0'`. A lead whose last 5 calls are Zero can't also have Interested as its last call.

FIX

```
ranked_calls AS (
    SELECT cl.lead_id, cl.started_at, d.name AS disposition,
           ROW_NUMBER() OVER (PARTITION BY cl.lead_id
                              ORDER BY cl.started_at DESC, cl.id DESC) AS rn
    FROM call_logs cl
    JOIN old_leads ol              ON ol.id = cl.lead_id
    JOIN lead_calls lc             ON lc.call_log_id = cl.id
    JOIN lead_call_dispositions d  ON d.id = lc.disposition_id
    WHERE cl.status NOT IN ('placing', 'call_failed')
),
last_5_zero AS (
    SELECT lead_id, MAX(started_at) AS last_call_at
    FROM ranked_calls
    WHERE rn <= 5
    GROUP BY lead_id
    HAVING COUNT(*) = 5
       AND BOOL_AND(COALESCE(TRIM(disposition) = 'Zero', false))
)
```

WHY IT WORKS

- `ROW_NUMBER` restarts at 1 for each lead (`PARTITION BY`); 1 is the newest call. `cl.id DESC` breaks ties.
- `HAVING` filters groups after `GROUP BY`; `WHERE` filters rows before.
- `COUNT(*) = 5` requires at least 5 calls. `BOOL_AND` is true only if every row passes.
- `BOOL_AND` skips NULLs, so `COALESCE(..., false)` makes a blank disposition break the streak.

Lesson: Rank, cut, group, test. And check the stored value before filtering on it.

20261029-06 Raw verification query showed non-Zero dispositions

SQL · **Wrong result** · window functions, WINDOW, verification

PROBLEM

The raw-data check listed rows with other dispositions, so it looked like the main query was wrong.

CAUSE

The check listed the last 5 calls of every lead, not only qualifying ones. The flags were window results, which can't be filtered in the same SELECT that creates them.

FIX

```
last_5 AS (
    SELECT rc.*,
           COUNT(*) OVER w AS calls_in_last_5,
           BOOL_AND(COALESCE(TRIM(rc.disposition) = 'Zero', false)) OVER w AS all_zero
    FROM ranked_calls rc
    WHERE rc.rn <= 5
    WINDOW w AS (PARTITION BY rc.lead_id)
)
SELECT l.lead_id, l.rn AS call_rank,
       l.started_at AT TIME ZONE 'Asia/Kolkata' AS call_time, l.disposition
FROM last_5 l
WHERE l.calls_in_last_5 = 5 AND l.all_zero
ORDER BY l.lead_id, l.rn;
```

WHY IT WORKS

- Adding `OVER (...)` to an aggregate gives each row its group's total without collapsing rows.
- `WINDOW w AS (...)` names the window once so every `OVER w` reuses it.
- Run order: FROM/JOIN, WHERE, GROUP BY, HAVING, window functions, SELECT, ORDER BY, LIMIT. So filter window results in the next step.
- Check: distinct `lead_id` in this output must equal the main query's row count.

Lesson: Calculate in one CTE, filter in the next. Always compare a summary with its raw rows.

20261029-07 Latest disposition is Zero, plus the distribution of the last 5

SQL · **How-to** · FILTER, STRING_AGG, two-level aggregation

PROBLEM

Needed leads whose latest call is Zero, with counts of each disposition in the last 5 and their order.

CAUSE

Counting per disposition and then per lead needs two levels of `GROUP BY`.

FIX

```
latest_zero AS (
    SELECT lead_id FROM last_5 WHERE rn = 1 AND disposition = 'Zero'
),
disp_counts AS (
    SELECT l.lead_id, l.disposition, COUNT(*) AS cnt
    FROM last_5 l JOIN latest_zero lz ON lz.lead_id = l.lead_id
    GROUP BY l.lead_id, l.disposition
),
per_lead AS (
    SELECT lead_id,
           SUM(cnt) AS calls_in_last_5,
           COALESCE(SUM(cnt) FILTER (WHERE disposition = 'Zero'), 0) AS zero_count,
           STRING_AGG(disposition || ' x' || cnt, ', ' ORDER BY cnt DESC, disposition) AS
    disposition_breakdown
    FROM disp_counts
    GROUP BY lead_id
)
-- sequence: STRING_AGG(disposition, ' -> ' ORDER BY rn)
```

WHY IT WORKS

- `FILTER (WHERE ...)` feeds only matching rows into an aggregate.
- `SUM` over zero rows is NULL, so wrap it in `COALESCE(..., 0)`.
- `STRING_AGG` skips NULLs, so replace NULL dispositions with `'(blank)'` first.
- Check: leads with `zero_count = 5` must equal entry 20261029-05's result.

Lesson: Count at the finest level first, then roll up.

20261029-08 Unreachable list, name normalisation, and last-10 pivot with call times

SQL · **How-to** · regexp_replace, VALUES, pivot

PROBLEM

Last 5 calls must all be one of 8 unreachable dispositions, with the last 10 shown as columns with IST times.

CAUSE

Names like `Did not Connected /Out of Service` break exact matches. `lead_call_dispositions.created_at` is when the option was set up in the CRM, not when a call happened.

FIX

```
target_dispositions AS (
    SELECT regexp_replace(lower(name), '[^a-z0-9]', '', 'g') AS disp_key
    FROM (VALUES ('Ringing/Busy'), ('Switched off'), ('Invalid Number'),
                 ('Did not Connected /Out of Service'), ('Disconnected by customer'),
                 ('Zero'), ('Didn''t connect'), ('No Answer')) AS v(name)
),
last_10 AS (
    SELECT lead_id, rn,
           started_at AT TIME ZONE 'Asia/Kolkata' AS call_time_ist,
           COALESCE(TRIM(disposition), '(blank)') AS disposition,
           regexp_replace(lower(COALESCE(disposition, '')), '[^a-z0-9]', '', 'g')
               IN (SELECT disp_key FROM target_dispositions) AS is_target
    FROM ranked_calls
    WHERE rn <= 10
)
-- pivot, one pair per call number:
-- MAX(l.disposition)   FILTER (WHERE l.rn = 1) AS call_1,
-- MAX(l.call_time_ist) FILTER (WHERE l.rn = 1) AS call_1_time
```

WHY IT WORKS

- `[^a-z0-9]` = any character that isn't a lowercase letter or digit; `'g'` replaces all of them.
- Both sides are normalised, so spacing, capitals and curly apostrophes don't matter.
- `'Didn''t connect'`: two single quotes inside an SQL string = one apostrophe.
- Each lead has one row per `rn`, so `MAX(...) FILTER (WHERE rn = N)` simply pulls that row into a column.

Lesson: Normalise both sides before comparing names. Call times come from `call_logs.started_at`.

20261029-09

column ol.client_id does not exist

SQL · **Error** · Metabase, column names

PROBLEM

Adding `ol.client_id` to the final SELECT failed.

CAUSE

`old_leads` is `SELECT a.*` from saved question 1717, and that question has no column called `client_id`.

FIX

```
-- See the real column names first
SELECT * FROM {{#1717-leads-analysis}} LIMIT 1;

-- Names with capitals or spaces need double quotes
SELECT ol."Client ID" ...

-- The right column turned out to be:
SELECT ol.customer_id ...
```

WHY IT WORKS

- Unquoted names are turned into lowercase, so `Client_ID` needs `"Client_ID"`.
- Questions built in Metabase's GUI with a join may prefix names, e.g. `"Clients__client_id"`.

Lesson: Look at the real column names before guessing one.

20261029-10 syntax error at or near "=" (Position 5128)

SQL · **Error** · JOIN syntax, aliases

PROBLEM

The query failed after adding trading-profile columns.

CAUSE

`left join ol.customer_id = ctp.customer_id` has no table name, no alias and no `ON`.

FIX

```
-- Wrong
left join ol.customer_id = ctp.customer_id

-- Right: JOIN <table> <alias> ON <condition>
LEFT JOIN customer_trading_profiles ctp ON ctp.customer_id = ol.customer_id

-- Give every AT TIME ZONE expression a name
ctp.last_txn_date AT TIME ZONE 'Asia/Kolkata' AS last_txn_date
```

WHY IT WORKS

- `Position: 5128` is the character count from the start of the query; look near it for the quoted word.
- Without an alias, an `AT TIME ZONE` column comes out named `timezone`.

Lesson: A join always reads JOIN <table> <alias> ON <condition>.

20261029-11 Leads repeated after joining customer_status_history

SQL · **Wrong result** · one-to-many join, DISTINCT ON, ARRAY_AGG

PROBLEM

Each lead appeared once per status change after the join.

CAUSE

The status table has many rows per customer. Joining it directly multiplies the lead rows.

FIX

```
csh_dedup AS (
    SELECT DISTINCT ON (customer_id, occurred_at)
           customer_id, occurred_at,
           NULLIF(TRIM(previous_status::text), '')                AS previous_status,
           COALESCE(NULLIF(TRIM(new_status::text), ''), '(blank)') AS new_status
    FROM customer_status_history
    WHERE customer_id IN (SELECT customer_id FROM old_leads)
    ORDER BY customer_id, occurred_at, created_at DESC, id DESC
),
status_summary AS (
    SELECT customer_id,
           (ARRAY_AGG(new_status ORDER BY occurred_at DESC))[1] AS rt_status,
           MAX(occurred_at)                                     AS rt_status_since,
           COUNT(*)                                             AS status_events,
           STRING_AGG(new_status || ' (' ||
                      TO_CHAR(occurred_at AT TIME ZONE 'Asia/Kolkata', 'DD Mon YYYY HH24:MI') || ')',
                      ' -> ' ORDER BY occurred_at)              AS status_journey
    FROM csh_dedup
    GROUP BY customer_id
)
-- final: LEFT JOIN status_summary ss ON ss.customer_id = ol.customer_id
```

WHY IT WORKS

- `DISTINCT ON (a, b)` keeps the first row per (a, b) in `ORDER BY` order; `created_at DESC` makes the newest row win.
- `(ARRAY_AGG(... ORDER BY ... DESC))[1]` = the latest value. PostgreSQL arrays start at 1.
- `::text` stops `TRIM` failing if the status column is an enum.
- Join on `ol.customer_id`, not `ctp.customer_id`, so leads with no trading profile keep their status.

Lesson: Collapse many-rows tables to one row per key before joining. Check COUNT(*) = COUNT(DISTINCT lead_id).

20261029-12

TypeError: Can only merge Series or DataFrame objects, a <class 'str'> was passed

pandas · **Error** · merge, latest row per key

PROBLEM

`pd.merge('df', 'nc_led', ...)` failed, and the result kept an extra `ClientID` column.

CAUSE

The quotes made the names text, not DataFrames. The merge also used the full ledger instead of the one-row-per-client balance, which would repeat leads.

FIX

```
nc_led['CreatedAt'] = pd.to_datetime(nc_led['CreatedAt'])

nc_led_balance = (nc_led.sort_values('CreatedAt', ascending=False)
                        .drop_duplicates(subset='ClientID', keep='first')[['ClientID', 'Balance']])

df_bal = pd.merge(df,
                  nc_led_balance.rename(columns={'ClientID': 'zynixx_client_id'}),
                  on='zynixx_client_id', how='left', validate='many_to_one')
```

WHY IT WORKS

- Sort newest first, then `drop_duplicates(keep='first')` = latest row per client (pandas' `DISTINCT ON`).
- Renaming the key and using `on=` leaves one key column. Or add `.drop(columns='ClientID')`.
- `validate='many_to_one'` raises an error if a client appears twice on the right.
- If `Balance` is empty everywhere, the key types differ: align them with `astype('string')`.

Lesson: Variables have no quotes. Merge one row per key, and let `validate=` check it.

20261029-13

Tag Active-but-unreachable clients as automatic traders

Excel · **How-to** · IF, COUNTIF, confidence levels

PROBLEM

The manager asked for clients who are Active but don't answer calls, tagged as automatic traders.

CAUSE

Being Active means different things depending on when they traded, so a confidence level was added.

FIX

```
AL  unanswered_call_days
=IF(COUNT(I2,K2,M2,O2,Q2)<5,"",1+(INT(I2)<>INT(K2))+(INT(K2)<>INT(M2))+(INT(M2)<>INT(O2))+(INT(
    O2)<>INT(Q2)))

AM  customer_tag
=IF(AG2="Active","Automatic Trader","")

AN  auto_trader_confidence
=IF(AM2="","",IF(OR(AL2=1,AE2<=Q2),"Low",IF(AE2>I2,"High","Medium")))

I = call_1_time (latest)   Q = call_5_time (streak start)   AE = last_txn_date   AG = rt_status
```

WHY IT WORKS

- `INT()` drops the time from a date-time; calls are sorted newest first, so each date change is a new day.
- Low: all 5 calls on one day, or last trade before the streak. High: traded after the latest missed call. Medium: in between.
- Result: 205 tagged (High 122, Medium 34, Low 49). Missed calls are 71% No Answer; only 1 Invalid Number.
- Blank confidence = not Active, by design (315 in Clients, 19,882 in Leads).

Lesson: Give a tag a confidence level when the evidence is uneven, and say what would confirm it.

Friday, 30 October 2026

Focus	Ledger checks in pandas, Excel formula fixes, and combining 60M+ rows of transaction files from two folders (CSV and XLSX) for the migration.
Outcome	21 entries: 8 errors, 6 wrong results and 7 how-tos. Built transactions_loader.py so the transaction files load without crashing the kernel.
Review	Repeated from day 1: a built-in used as a name (all), duplicate column names (four amount columns), and latest-row-per-key sorted by the wrong column. Focus tomorrow: PG40 Day 1, exercises 1 to 10.
Cards	`20261030-01` to `20261030-21`

20261030-01 Select ledger rows for yesterday

pandas · **How-to** · dates, half-open range, sort_values

PROBLEM

Needed only yesterday's ledger rows. The code also ran `led.sort_values(...)` on its own line.

CAUSE

A date filter needs a start and an end. `sort_values` returns a sorted copy, so without `led = ...` the sort is thrown away.

FIX

```
led = led.sort_values('CreatedAt', ascending=False)      # assign it back

today = pd.Timestamp.today().normalize()      # today 00:00:00
yesterday = today - pd.Timedelta(days=1)

led_yesterday = led[(led['CreatedAt'] >= yesterday) & (led['CreatedAt'] < today)]
```

WHY IT WORKS

- `>=` start and `<` end (a half-open range) covers every second, including 23:59:59.5.
- Each condition in brackets, joined with `&`; plain `and` doesn't work on columns.
- If `led['CreatedAt'].max()` is older than yesterday, 0 rows is the correct answer.
- UTC times are 4 hours behind Dubai: convert with `tz_localize('UTC').tz_convert('Asia/Dubai')` first.

Lesson: Use >= start and < end for date ranges, and always assign sort_values back.

20261030-02 The combined ledger seemed to stop at the 27th

Polars · **Wrong result** · ledger_loader, diagnostics, Excel-saved CSV

PROBLEM

`get_final_ledger()` looked like it had no rows after the 27th, although files for 28 and 29 Oct existed.

CAUSE

The loader parses `2026-10-29T23:59:53.103Z` correctly. The likely cause was viewing an unsorted `head()`, or a file missing from the append folder or saved through Excel.

FIX

```
print(led['CreatedAt'].max())          # step 1: is the data really missing?

summary = lambda lf: lf.select(
    pl.col("CreatedAt").min().alias("first"),
    pl.col("CreatedAt").max().alias("last"),
    pl.col("CreatedAt").null_count().alias("unparsed_dates"),
    pl.len().alias("rows"),
).collect()
print(summary(scan_base_ledger()))
print(summary(scan_append_files()))
```

WHY IT WORKS

- Check the data before changing the code: `max()` answers "is it there?" in one line.
- `unparsed_dates` above 0 means some date text didn't match the format, usually after Excel saved it.
- `trade_id` shown as `2.61029E+17` means the file was opened in Excel; if saved, the IDs are rounded for good.

Lesson: Measure first (min, max, nulls, rows per source), then decide what's broken.

20261030-03 SUMIFS returned 0 for negative balance protection

Excel · **Wrong result** · SUMIFS, $ references, COUNTIF

PROBLEM

`=SUMIFS(...!I:I, ...!E:E, Sheet2!D3, ...!C:C, Sheet2!A4)` gave 0 down the column.

CAUSE

`D3` had no `$`, so it moved to `D4`, `D5`... when copied down. The ledger type may also be spelled differently, or there may be no such entries that day.

FIX

```
=SUMIFS('direct_ledger_report_filtered ('!$I:$I,
        'direct_ledger_report_filtered ('!$E:$E, D$3,
        'direct_ledger_report_filtered ('!$C:$C, $A4)

=COUNTIF('direct_ledger_report_filtered ('!E:E, D3)            exact matches
=COUNTIF('direct_ledger_report_filtered ('!E:E, "*NEGATIVE*")   any spelling
```

WHY IT WORKS

- `D$3` fixes the row (a header); `$A4` fixes the column (a label); `$I:$I` fixes both.
- `SUMIFS` ignores capitals but not underscores or trailing spaces. `*` in a criterion = any characters.
- Easier: put `ledger_type` in the pivot table's Columns area to get every type at once.

Lesson: Anchor headers with D$3 and labels with $A4 before copying a formula.

20261030-04 Extract the number from BR575346: ****

Excel · **How-to** · REGEXEXTRACT, MID, FIND

PROBLEM

Needed `575346` out of IDs like `BR575346: ****`, where the letter prefix varies in length.

CAUSE

The digits start at a different position for one-letter (`S75577`) and two-letter prefixes.

FIX

```
=REGEXEXTRACT(A2, "[0-9]+")                                  Microsoft 365

=MID(A2, MIN(FIND({0,1,2,3,4,5,6,7,8,9}, A2&"0123456789")),     older Excel
     FIND(":", A2) - MIN(FIND({0,1,2,3,4,5,6,7,8,9}, A2&"0123456789")))
```

WHY IT WORKS

- `[0-9]+` = one or more digits in a row; `--` in front turns the text into a number.
- `MIN(FIND({0..9}, ...))` = position of the first digit; adding `"0123456789"` stops `FIND` erroring.
- pandas: `str.extract(r'(\d+)', expand=False)`. PostgreSQL: `substring(col FROM '[0-9]+')`.

Lesson: A regex finds a pattern wherever it sits; position-based formulas break when prefixes vary.

20261030-05 The formula shows as text instead of calculating

Excel · **Error** · cell format, Show Formulas

PROBLEM

`=REGEXEXTRACT(A2, "[0-9]+")` stayed visible as text in the cell.

CAUSE

The cell was formatted as Text, so Excel stored the formula as plain text.

FIX

```
1. Select the cells -> Home -> Number format -> General
2. Re-enter: F2, then Enter (changing the format alone doesn't recalculate)
3. Many cells: Ctrl+H, replace = with =, Replace All
```

WHY IT WORKS

- Other causes: Show Formulas is on (Formulas -> Show Formulas), or a space or apostrophe before the `=`.
- If it then shows `#NAME?`, the Excel version has no `REGEXEXTRACT`; use the `MID` version.

Lesson: A formula that won't calculate is usually a Text-formatted cell.

20261030-06 UNIQUE(FILTER(..., OR(...))) doesn't filter

Excel · **Wrong result** · FILTER, OR logic, arrays

PROBLEM

`=UNIQUE(FILTER(dgrdReport!G:G, OR(D:D="Buy", D:D="Sell")))` gave an error or wrong rows.

CAUSE

`OR()` returns one TRUE/FALSE for the whole column; `FILTER` needs one per row.

FIX

```
=UNIQUE(FILTER(dgrdReport!G:G,
               (dgrdReport!D:D="Buy") + (dgrdReport!D:D="Sell"),
               "No matches"))

=UNIQUE(FILTER(dgrdReport!G:G, ISNUMBER(MATCH(dgrdReport!D:D, {"Buy","Sell"}, 0)), "No matches"))
```

WHY IT WORKS

- `+` turns TRUE/FALSE into 1/0 row by row; `FILTER` keeps non-zero rows. `*` means AND.
- `"No matches"` replaces the `#CALC!` error when nothing matches.
- Whole columns like `G:G` are slow; fixed ranges like `G2:G50000` are faster.

Lesson: Inside FILTER: + for OR, * for AND. Never OR() or AND().

20261030-07 KeyError: 'ClientID' in groupby

pandas · **Error** · variable names, columns

PROBLEM

`df.groupby('ClientID', as_index=False)['CreatedAt'].max()` raised `KeyError: 'ClientID'`.

CAUSE

`df` had just been replaced by `pd.read_excel(...)`, so it held the traded-IDs sheet, not the ledger.

FIX

```
print(df.columns.tolist())                    # what does df actually hold?

last_dates = led.groupby('ClientID', as_index=False)['CreatedAt'].max()

df.columns = df.columns.str.strip()           # if names have hidden spaces
df = df.reset_index()                         # if ClientID became the index
```

WHY IT WORKS

- A `KeyError` on a column means that exact name isn't in `df.columns`: wrong table, spaces, capitals, or index.
- `pd.read_excel(path, sheet_name='Sheet1')` reads one tab; `pd.ExcelFile(path).sheet_names` lists them.

Lesson: Give every table its own name (led, traded_ids); reusing df hides which table you have.

20261030-08 Fill Zynixx Balance and Last Trade with .map()

pandas · **How-to** · map, lookup, set_index

PROBLEM

The traded-IDs sheet had empty `Zynixx Balance` and `Zynixx Last T` columns to fill from two small tables.

CAUSE

`merge` would add new key and value columns. `.map()` fills an existing column and keeps the row order.

FIX

```
bal_map  = balance.drop_duplicates('ClientID').set_index('ClientID')['Balance']
date_map = last_dates.drop_duplicates('ClientID').set_index('ClientID')['CreatedAt']

key = df['Zynixx Id'].astype(str).str.strip()
df['Zynixx Balance'] = key.map(bal_map)
df['Zynixx Last T']  = key.map(date_map)
```

WHY IT WORKS

- `set_index(key)[col]` turns a table into a lookup: labels are IDs, values are the answers.
- IDs not found (like `No New Id`) become NaN; `df` keeps its 212 rows.
- Duplicate IDs in the lookup make `.map()` fail; `drop_duplicates` prevents it.

Lesson: To fill a column from another table by key, use .map() on a one-row-per-key lookup.

20261030-09 Latest balance taken from a random row

pandas · **Wrong result** · drop_duplicates, latest row per key

PROBLEM

`led_req.sort_values('ClientID').drop_duplicates(subset='ClientID', keep='first')` gave balances that weren't the latest.

CAUSE

Sorting by `ClientID` groups each client's rows but leaves them in no date order, so `keep='first'` picks any row.

FIX

```
balance = (led_req.sort_values('CreatedAt', ascending=False)
                  .drop_duplicates(subset='ClientID', keep='first')
                  .reset_index(drop=True)[['ClientID', 'Balance']])
```

WHY IT WORKS

- "Latest row per key" = sort by the DATE, newest first, then keep the first row per key.
- Same idea as `DISTINCT ON (key) ... ORDER BY key, date DESC` in PostgreSQL.

Lesson: keep='first' is only as good as the sort before it: sort by the column that defines 'first'.

20261030-10 KeyError: Index([Ellipsis], dtype='object')

pandas · **Error** · placeholder code

PROBLEM

Running a comparison block failed inside `drop_duplicates`.

CAUSE

The block included `drop_duplicates(...)` with a literal `...`, written as a placeholder. Python treats `...` as a real value (`Ellipsis`) and looked for a column with that name.

FIX

```
# Run only the real line, never the illustrative one:
balance = (led_req.sort_values('CreatedAt', ascending=False)
                  .drop_duplicates(subset='ClientID', keep='first'))
```

WHY IT WORKS

- `...` is valid Python, so it doesn't fail until the function uses it.
- When a snippet shows a "before" and "after", copy only the "after".

Lesson: Read a code block before running it; skip lines marked as examples of what not to do.

20261030-11 Add a result sheet to the original workbook

pandas · **How-to** · ExcelWriter, openpyxl

PROBLEM

Needed the finished `df` saved as a sheet called `result` in the original Excel file.

CAUSE

`df.to_excel(path)` alone overwrites the whole workbook.

FIX

```
with pd.ExcelWriter(path, engine='openpyxl', mode='a', if_sheet_exists='replace') as writer:
    df.to_excel(writer, sheet_name='result', index=False)
```

WHY IT WORKS

- `mode='a'` adds to the file and keeps the other sheets.
- `if_sheet_exists='replace'` overwrites `result` on reruns instead of creating `result1`.
- Close the file in Excel first (`PermissionError`), and note `openpyxl` can drop pivot tables and slicers.

Lesson: Append a sheet with ExcelWriter(mode='a'); write a new file when the workbook has pivots.

20261030-12 Combining every file in a folder

pandas · **Error** · os.listdir, concat, built-in names

PROBLEM

The loop started with `all = []`, called `pd.read_csv()` with no path, and created an empty table on every pass.

CAUSE

`all` is a built-in function; `read_csv` needs the full path; `.xlsx` needs `read_excel`.

FIX

```
frames = []
for file in os.listdir(folder):
    full_path = os.path.join(folder, file)
    if file.startswith('~$'):
        continue
    if file.lower().endswith('.csv'):
        part = pd.read_csv(full_path, dtype=text_cols)
    elif file.lower().endswith('.xlsx'):
        part = pd.read_excel(full_path)
    else:
        continue
    part['source_file'] = file
    frames.append(part)

combined = pd.concat(frames, ignore_index=True)
```

WHY IT WORKS

- Collect pieces in a list and `concat` once; it is much faster than growing a table file by file.
- `os.listdir` returns names only, so `os.path.join` builds the path.
- `~$` files are Excel lock files and can't be read.

Lesson: Same lesson as min and max yesterday: never use a built-in name (all, list, sum) for a variable.

20261030-13 DtypeWarning: Columns (21: expiryDate) have mixed types

pandas · **Error** · dtype, read_csv, mixed types

PROBLEM

`read_csv` warned about `expiryDate`, `comment` and `scriptParent` in several files.

CAUSE

Big files are read in chunks and each chunk's type is guessed separately. Blank chunks look like numbers and others hold text, so the column ends up mixed.

FIX

```
text_cols = {'expiryDate': str, 'comment': str, 'scriptParent': str}
part = pd.read_csv(full_path, dtype=text_cols)

combined['expiryDate'] = pd.to_datetime(combined['expiryDate'], errors='coerce')
combined['expiryDate'].map(type).value_counts()      # check what a column holds
```

WHY IT WORKS

- A mixed column breaks sorting (`'<' not supported between 'str' and 'float'`) and merges.
- `low_memory=False` only hides the warning; the values are still mixed.

Lesson: Read mostly-empty or mixed columns as text, then convert them once on purpose.

20261030-14 MemoryError: Unable to allocate 487 MiB ... shape (1, 63807297)

pandas · **Error** · memory, usecols, concat

PROBLEM

`pd.concat(frames)` failed after reading every transaction file.

CAUSE

63,807,297 rows × 8 bytes = 487 MiB per number column. With ~30 columns the table needs 15+ GB, and `concat` holds the pieces and the copy at once.

FIX

```
part = pd.read_csv(full_path, usecols=keep_cols, dtype=text_cols)   # only needed columns
part = part[part['clientId'].isin(ids)]                              # only needed rows
frames.append(part)
...
combined = pd.concat(frames, ignore_index=True)
del frames                                                           # free the pieces
```

WHY IT WORKS

- Estimate first: rows × columns × 8 bytes for numbers; text costs more.
- Cutting columns while reading saves the most; filtering each file saves the rest.
- For this size, Polars (lazy, parallel) is the better tool.

Lesson: Size the data before loading it; read only the columns and rows you need.

20261030-15 'LazyFrame' object has no attribute 'to_pandas'

Polars · **Error** · LazyFrame, collect

PROBLEM

`lf = lf.to_pandas()` failed.

CAUSE

A LazyFrame is a plan of what to read, not data. Only a collected DataFrame has `.to_pandas()`.

FIX

```
lf.collect_schema().names()     # column names, no data read
lf.head(5).collect()            # first rows only
lf.select(pl.len()).collect()   # row count

result_pd = lf.select(cols).filter(cond).collect().to_pandas()
```

WHY IT WORKS

- `.collect()` runs the plan; `.to_pandas()` then converts the result.
- Don't reuse `lf` for a pandas table; keep `lf` for the plan and new names for data.

Lesson: Lazy = plan. collect() = load. Cut the plan down before collecting.

20261030-16 Kernel crashed on lf.collect()

Polars · **Error** · memory, streaming, sink

PROBLEM

`result = lf.collect()` then `result.to_pandas()` crashed the Jupyter kernel; every variable was lost.

CAUSE

It loaded all 63.8M+ rows and every column, then tried to make a second full copy for pandas.

FIX

```
needed = (lf.select(['clientId', 'symbol', 'time'])
            .filter(pl.col('clientId').is_in(list(ids)))
            .collect(engine='streaming'))                  # older Polars: streaming=True
print(needed.shape, round(needed.estimated_size('gb'), 2), 'GB')

lf.sink_csv(r"C:\...\transactions_combined.csv")       # everything, to disk, in batches
```

WHY IT WORKS

- `engine='streaming'` processes in batches; `group_by(...).agg(...)` before collect returns only totals.
- `sink_csv` / `sink_parquet` write every row without holding it in memory, which suits `\copy` into PostgreSQL.
- Check `estimated_size('gb')` before `.to_pandas()`; converting needs that much again.

Lesson: Never collect a whole big plan. Select, filter, or sink to disk.

20261030-17 Append the ark folder (CSV and XLSX) to the transactions

Polars · **How-to** · read_excel, diagonal_relaxed, source_file

PROBLEM

The ark append folder had both `.csv` and `.xlsx` files to add to the transaction plan.

CAUSE

Polars scans CSVs lazily but must read Excel files fully; both then have to join one plan.

FIX

```
def read_xlsx_file(f):
    return (pl.read_excel(f, infer_schema_length=10_000)     # pip install fastexcel
              .lazy()
              .with_columns(pl.lit(f.name).alias('source_file')))

parts = [scan_csv_file(f) for f in trans_csv + ark_csv] + [read_xlsx_file(f) for f in ark_xlsx]
lf = pl.concat(parts, how='diagonal_relaxed')
```

WHY IT WORKS

- `.lazy()` turns a loaded Excel table into a LazyFrame; each `.xlsx` is at most ~1M rows.
- `diagonal_relaxed` matches columns by name and widens differing types (a date vs text becomes text).
- `pl.lit(f.name)` tags each row with its file, so you can count rows per source.

Lesson: Tag every row with its source file whenever you combine files.

20261030-18 transactions_loader.py: a module that can't crash the kernel

Polars · **How-to** · module, select before collect

PROBLEM

`get_final_ledger(as_pandas=True)` never crashed, but loading the transactions did.

CAUSE

The ledger loader keeps 6 columns before collecting, so Polars reads only those. The transactions code collected 30+ columns.

FIX

```
from transactions_loader import list_columns, check_columns, count_rows, get_transactions

check_columns()                                  # files whose columns differ
count_rows(by_file=True)                         # rows per file, nothing loaded
df = get_transactions(columns=['ticketId', 'time', 'accountId', 'amount'],
                      where=pl.col('accountId').is_in(list(ids)),
                      as_pandas=True)
```

WHY IT WORKS

- `columns` is required, so the full table can't be loaded by accident.
- Results over `MAX_PANDAS_GB` (4 GB) raise a clear error instead of crashing.
- `export_transactions(path)` streams everything to `.parquet` or `.csv` for the migration.

Lesson: Build the safe habit into the tool: make the big, dangerous option impossible by default.

20261030-19 Aligning ark columns gave four amount columns

pandas · **Wrong result** · rename, normalised names, clash check

PROBLEM

After renaming ark columns to the main names, the result had `amount` four times: `1.00, 37372.0, 1.00, 37372.0`.

CAUSE

Two ark columns normalised to `amount`, and selecting `[list(rename.values())]` asked for `amount` twice, so each copy appeared twice. The code had no clash check.

FIX

```
def norm(name):
    return re.sub(r'[^a-z0-9]', '', str(name).lower())      # 'Ticket Id' -> 'ticketid'

targets = pd.Series(rename)
clashes = targets[targets.duplicated(keep=False)]
if not clashes.empty:
    raise ValueError("Same target name:\n" + clashes.to_string())

ark_clean = df[list(rename)].rename(columns=rename)          # select originals, then rename
```

WHY IT WORKS

- Lowercase alone fails: `Ticket Id` -> `ticket id`, not `ticketId`. Removing spaces and symbols matches them.
- Names that differ in words (`Account` -> `accountId`, `Open Com` -> `openCommission`) are mapped by hand.
- Each ark file has different headers (31 vs 14 columns), so align each file on its own.

Lesson: After any rename, check that no two columns share a name.

20261030-20 Match the main file's columns and types exactly (minus id)

pandas · **How-to** · reindex, Int64, dtypes

PROBLEM

The cleaned ark table had 28 of the 30 target columns, in a different order, with `ticketId` as `41672978.0`.

CAUSE

Missing columns, different order, and blanks forcing whole numbers into decimals.

FIX

```
target_cols = [c for c in major.columns if c != 'id']
major_final = major.drop(columns='id')
ark_final = ark_clean.reindex(columns=target_cols)       # adds fileReferenceId, location as blank
assert list(ark_final.columns) == list(major_final.columns)

for df_ in (major_final, ark_final):
    for c in ['ticketId', 'openPosition']:
        df_[c] = pd.to_numeric(df_[c], errors='coerce').astype('Int64')
```

WHY IT WORKS

- `reindex(columns=...)` sets the order and adds missing columns as blanks in one step.
- `Int64` (capital I) allows blanks and keeps whole numbers.
- Compare types side by side: `pd.DataFrame({'major': major_final.dtypes, 'ark': ark_final.dtypes})`.

Lesson: Same names is not enough: make the types match before appending.

20261030-21 Times show as 59:48.4 (date and hour lost)

pandas · **Wrong result** · Excel, CSV, data loss

PROBLEM

`ark_final['time']` held values like `59:48.4` and `openDate` like `28:14.3`.

CAUSE

The CSV was opened in Excel and saved. Excel displays times with fractions as minutes:seconds and saves the display, so the date and hour are gone from the file.

FIX

```
short = ark_clean['time'].astype(str).str.fullmatch(r'\d{1,2}:\d{2}(\.\d+)?')
ark_clean.loc[short, 'source_file'].value_counts()            # which files are damaged
print(ark_clean.duplicated(subset='ticketId').sum())          # same rows in csv AND xlsx?

t = ark_final['time'].astype(str).str.strip()
ark_final['time'] = pd.to_datetime(t.where(~t.str.fullmatch(r'\d{1,2}:\d{2}(\.\d+)?')),
                                   format='ISO8601', errors='coerce')     # blank, don't guess
```

WHY IT WORKS

- The same rows appeared with full times in another ark file, so a clean copy likely exists.
- If the same data is in a `.csv` and an `.xlsx`, every row is loaded twice; removing the damaged file fixes both.
- Never save a CSV from Excel; look at it with `pd.read_csv(path, nrows=20)` instead.

Lesson: Keep original downloads untouched. Excel rewrites what it saves.

Saturday, 31 October 2026

Focus	Rebuilding the Stringee + Uno dialer report as a fast, tested script; making it multi-day; the Uno API token; client trading days.
Outcome	12 entries: 3 errors, 3 wrong results and 6 how-tos. dialer_report.py matched the old notebook cell for cell on Stringee data, fixed three old bugs, and runs a test day in 2.1 s instead of 7.9 s.
Review	Repeated: debugging code before checking inputs (the report read an old folder), and trusting output from an older run (the trading-days file). Focus tomorrow: PG40 Day 1, exercises 1 to 5.
Cards	`20261031-01` to `20261031-12`

20261031-01 FILTER rejected: 'BILL' in single quotes

Excel · **Error** · quotes, FILTER, UNIQUE

PROBLEM

`=unique(FILTER('31 oct 2026'!C:C,'31 oct 2026'!E:E='BILL'))` wouldn't work.

CAUSE

Excel uses single quotes only around sheet names. Text values need double quotes.

FIX

```
=UNIQUE(FILTER('31 oct 2026'!C:C, '31 oct 2026'!E:E="BILL", "No matches"))
```

WHY IT WORKS

- `'31 oct 2026'!` = a sheet name with spaces; `"BILL"` = text.
- `"No matches"` replaces the `#CALC!` error when nothing matches.
- The match ignores capitals but not trailing spaces.

> **Lesson:** Single quotes for sheet names, double quotes for text.

20261031-02 Organised, faster Stringee CDR combine script

pandas · **How-to** · pathlib, calamine, xlsxwriter, # %% cells

PROBLEM

The daily script used `os.chdir`, unused imports, a `> 2` file check and slow Excel engines.

CAUSE

`> 2` let 0 or 1 file through and then failed on `xlsx_files[1]`; Excel lock files counted as inputs.

FIX

```
files = sorted(f for f in INPUT_DIR.glob("*.xlsx") if not f.name.startswith("~$"))
if len(files) != 2:
    raise ValueError(f"Expected exactly 2 .xlsx files, found {len(files)}: {[f.name for f in files]}")

def read_xlsx(path):
    try:
        return pd.read_excel(path, engine="calamine")     # pip install python-calamine
    except ImportError:
        return pd.read_excel(path)
```

WHY IT WORKS

- `Path` objects carry the full folder, so `os.chdir` isn't needed.
- `calamine` reads and `xlsxwriter` writes much faster; the helper falls back if missing.
- `format="ISO8601"` reads times with or without fractions of a second.
- `# %%` cells in a `.py` file convert to `.ipynb` with VS Code or `jupytext --to notebook`.

> **Lesson:** Make a file-count check list the files it found; it turns a crash into an answer.

20261031-03 Read IDs as text; convert dates and durations after

pandas · **How-to** · dtype, to_timedelta, [h]:mm:ss

PROBLEM

Stringee files mix real date-times, `00:05:40` durations and 12-digit phone numbers.

CAUSE

A blank cell turns a phone column into floats (`9.19e+11`), and Excel durations arrive as time-of-day values.

FIX

```
df = pd.read_excel(path, dtype={"ID": str, "Customer number": str, "Hotline": str})
for c in ["Queue duration", "Answer duration", "Hold duration"]:
    df[c] = pd.to_timedelta(df[c].astype(str).str.strip(), errors="coerce")

# writing back: numbers as day fractions + a duration format
out[c] = out[c].dt.total_seconds() / 86_400
writer.sheets["CDR"].set_column(i, i, 14, writer.book.add_format({"num_format": "[h]:mm:ss"}))
```

WHY IT WORKS

- Only IDs need `dtype` at read time; afterwards the float damage is done.
- pandas writes timedeltas as `0.003935`; with `[h]:mm:ss` Excel shows `0:05:40` and `SUM` still works.
- Writing numbers first matters: pandas' own timedelta cells carry a `0` format that overrides the column's.

> **Lesson:** Fix types at the boundary: text IDs on the way in, display formats on the way out.

20261031-04 Old dialer notebook: Uno start times lost and durations zeroed

pandas · **Wrong result** · regression test, Uno, timedelta

PROBLEM

In the old notebook every Uno call landed in the `20+` slot, with no gaps, Total_Duration 00:00:00 and Login Hours 01:00:00.

CAUSE

The date was glued to Uno's full date-time (`'2026-10-30 2026-10-30 09:12:00'`, which can't parse), and a later step set anything not stored as text to `Timedelta(0)`.

FIX

```
uno_start = pd.to_datetime(
    uno["Call Date"].dt.strftime("%Y-%m-%d") + " " + uno["Call Time"].astype(str).str.strip().str.
    upper(),
    format="%Y-%m-%d %I:%M:%S %p", errors="coerce")
uno_calls["Total Call Duration"] = to_td(uno["Call Duration"])     # handles text and timedeltas
```

WHY IT WORKS

- Found by running a faithful copy of the old notebook beside the new script on the same test days.
- Stringee results matched cell for cell; only Uno rows differed, which pointed straight at the bugs.
- A third bug: `int(mins * 60)` cut 1 second from about 2% of hourly talk times (100 s became 99 s).

> **Lesson:** Before replacing a report, run old and new on the same data and explain every difference.

20261031-05 Call gaps with groupby().shift() instead of a loop

pandas · **How-to** · groupby, shift, vectorise

PROBLEM

The gap calculation looped row by row with `.loc`, the slowest part of the notebook.

CAUSE

Each call needs the previous call's start and duration for the same CRE and day.

FIX

```
dialers = dialers.sort_values(["Date", "CRM ID", "Call Start Time"], kind="mergesort").reset_index(
    drop=True)
g = dialers.groupby(["Date", "CRM ID"], sort=False)
prev_start = g["Call Start Time"].shift()
prev_end = prev_start + g["Total Call Duration"].shift()
gap = (dialers["Call Start Time"] - prev_end).clip(lower=pd.Timedelta(0))
```

WHY IT WORKS

- `shift()` moves values down one row inside each group, so the first call of a day gets nothing.
- Sorting first is essential; `mergesort` keeps ties in a fixed order.
- Summary sums use pre-computed columns (`talk_s.where(conn, 0)`) instead of per-group lambdas.
- Test day of 8,162 calls: 7.9 s before, 2.1 s after (0.6 s without saving raw copies).

> **Lesson:** "Previous row in the same group" is a groupby shift, not a loop.

20261031-06 Grouper for 'Date' not 1-dimensional

pandas · **Error** · pivot_table, keys

PROBLEM

After adding `"Date"` to the hourly pivot keys, the Calls pivot failed.

CAUSE

That pivot counted `values="Date"`, so `Date` was both a group key and the value counted.

FIX

```
keys = ["Date", "CRM ID", "Employee code", "Full Name", "Pool", "TL"]
calls = dialers.pivot_table(index=keys, columns="Interval", values="Call Status",
                            aggfunc="count", fill_value=0)
```

WHY IT WORKS

- Count any column that is never blank; `Call Status` always has a value.
- The same report then merges Login Hours on `["CRM ID", "Date"]` so days don't repeat rows.

> **Lesson:** A column can be a key or a value in one pivot, not both.

20261031-07 Multi-day reports with checks that fail loudly

pandas · **How-to** · concat, drop_duplicates, consistency check

PROBLEM

The report needed to cover any number of days from several exports, some overlapping.

CAUSE

The old version accepted exactly 2 Stringee files and 1 Uno file and assumed a single day.

FIX

```
uno = pd.concat([pd.read_csv(f, dtype=uno_text) for f in uno_files], ignore_index=True)
uno = uno.drop_duplicates(subset="Call Id")

check = hourly[on + ["Total Calls"]].merge(summary[on + ["Total_Dialed_Calls"]], on=on, how="left")
print("Hourly totals match the summary:", (check["Total Calls"] == check["Total_Dialed_Calls"]).all())
print("Calls per day:", dialers.groupby(dialers["Date"].dt.strftime("%Y-%m-%d")).size().to_dict())
```

WHY IT WORKS

- 3-day test: each day matched a single-day run; 50 overlapping Uno rows removed.
- Two reports from the same calls must agree; a mismatch means a join or filter went wrong.
- File names show the range: `Summary_2026-10-28_to_2026-10-30.csv`.

Lesson: Print counts per day and cross-check totals before anyone opens the file.

20261031-08 Report showed only the latest date: wrong input folder

pandas · **Wrong result** · inputs, folders, debugging order

PROBLEM

The summary only had 28 Oct, although an export from 1 Oct existed.

CAUSE

`STRINGEE_DIR` pointed to `Documents\Dialer\st_daily`, holding only the 28th. The full export was in `Videos\Office\Dialer\CDR\Stringee`.

FIX

```
for f, p in zip(st_files, st_parts):
    t = pd.to_datetime(p["Start time"], format="%m/%d/%Y %I:%M:%S %p", errors="coerce")
    print(f"{f.parent.name}/{f.name}: {len(p):,} rows | {t.min()} -> {t.max()}")
```

WHY IT WORKS

- 11 rounds were spent on date parsing and header theories; tests ruled both out.
- The file list printed on every run (`20261029_052117.xlsx` ...) held the clue from the start.
- Printing the folder name beside each file shows a wrong folder at a glance.

Lesson: Check which files are read, from which folder, and their date ranges before touching the code.

20261031-09 Parse 10/28/2026 09:34:25 PM safely

pandas · **How-to** · to_datetime, format, month-first

PROBLEM

Stringee shows start times as `10/28/2026 09:34:25 PM`.

CAUSE

A guessing parser can swap day and month: in testing it read `11/03/2026` as 11 March.

FIX

```
st_raw["Start time"] = pd.to_datetime(st_raw["Start time"], format="%m/%d/%Y %I:%M:%S %p",
    errors="coerce")
unread = st_raw["Start time"].isna().sum()
if unread:
    raise ValueError(f"{unread} Stringee start times could not be read")
```

WHY IT WORKS

- `%m/%d/%Y` = month first; `%I` + `%p` = 12-hour clock with AM/PM.
- Real Excel dates that only display this way pass through unchanged.
- `errors="coerce"` + the `unread` check stops the run instead of dropping rows silently.

Lesson: Give date formats explicitly and make failures stop the run.

20261031-10 Use the newest team list in a folder

pandas · **How-to** · pathlib, st_mtime

PROBLEM

The team list file name changes with every download.

CAUSE

A hard-coded file name breaks as soon as a new export arrives.

FIX

```
team_files = sorted(f for f in TEAM_DIR.iterdir()
                    if f.suffix.lower() in (".csv", ".xlsx") and not f.name.startswith("~$"))
team_file = max(team_files, key=lambda f: f.stat().st_mtime)
ref = (pd.read_csv(team_file, dtype={"Dialer Name": str}) if team_file.suffix.lower() == ".csv"
       else read_xlsx(team_file, dtype={"Dialer Name": str}))
```

WHY IT WORKS

- `st_mtime` = last-modified time; `max` picks the newest.
- Only one list is used: combining old and new lists would duplicate CRM IDs.
- Same day: `EXCLUDE_STRINGEE_END_CODES = []` keeps CAN_NOT_MAKE_CALL as dialed, not connected.

Lesson: Point settings at folders, then pick the file by rule and print which one was used.

20261031-11 KeyError on os.environ: token used as the variable name

Python · **Error** · environment variables, setx, secrets

PROBLEM

`TOKEN = os.environ["<the token>"]` raised `KeyError`.

CAUSE

`os.environ[...]` looks a variable up by NAME. The token's value was passed, and no variable has that name.

FIX

```
# once, in a terminal, then fully restart VS Code:
#   setx UNO_API_TOKEN "your-token"
TOKEN = os.environ["UNO_API_TOKEN"]
print("UNO_API_TOKEN" in os.environ)      # True
```

WHY IT WORKS

- Programs read environment variables only when they start, so restart the editor after `setx`.
- A token pasted into chat is exposed: regenerate it and run `setx` again.
- The API endpoint is still to come from the Uno panel; Uno looks like DeepCall (unconfirmed).

Lesson: Secrets live in environment variables, read by name; never in notebooks or chats.

20261031-12 Over 31 trading days in a month

pandas · **Wrong result** · nunique, normalize, assert

PROBLEM

The client report showed 155, 229 and 446 "days" in single months.

CAUSE

It still counted BILL rows. The file came from the old pivot: no `Total Trading Days` column existed.

FIX

```
bill_led["Trade Date"] = bill_led["CreatedAt"].dt.normalize()
pivot = bill_led.pivot_table(index="ClientID", columns="Month", values="Trade Date",
                             aggfunc="nunique", fill_value=0)
assert pivot.max().max() <= 31, "Still counting bills, not days"
```

WHY IT WORKS

- `.dt.normalize()` drops the time; `nunique` counts distinct dates.
- The `assert` stops an impossible result before it reaches Excel.
- Close the Excel file before exporting, or the old version stays on disk.

Lesson: Add a check for the impossible (more than 31 days) so wrong logic can't ship quietly.

Monday, 2 November 2026

Focus	Lead sources for depositors, October traders by manager and agent, a per-client report of Sales calls after conversion, and IDs sharing an IP in Metabase. Also built the 115-exercise mastery workbook.
Outcome	10 entries: 8 how-tos and 2 wrong results. Every join that could repeat rows was guarded before it could cause a problem.
Review	Most extra rounds came from unclear output columns (the 30-day request took four rounds). Focus tomorrow: mastery workbook exercises 1 to 15, then progress().
Cards	`20261102-01` to `20261102-10`

`20261102-01`

Lead source for depositing customers

SQL · **How-to** · DISTINCT ON, self-join, COALESCE

PROBLEM

Needed Lead Source 1 and 2 for every customer with `total_deposit_count > 0`.

CAUSE

A customer can have several leads, so joining `leads` directly repeats depositors. The source has a parent source in the same `sources` table.

FIX

```
WITH lead_source AS (
    SELECT DISTINCT ON (l.customer_id) l.customer_id, l.id AS lead_id,
           COALESCE(s2.name, 'Unknown') AS lead_source_1, s1.name AS lead_source_2
    FROM leads l
    LEFT JOIN sources s1 ON l.source_two_id = s1.id
    LEFT JOIN sources s2 ON s1.parent_id = s2.id
    WHERE l.customer_id IS NOT NULL AND l.deleted_at IS NULL
    ORDER BY l.customer_id, l.date_of_lead, l.created_at, l.id
)
SELECT ctp.*, ls.lead_source_1, ls.lead_source_2
FROM customer_trading_profiles ctp
LEFT JOIN lead_source ls ON ls.customer_id = ctp.customer_id
WHERE ctp.total_deposit_count > 0;
```

WHY IT WORKS

- `sources s1` and `sources s2` are the same table under two aliases: the source and its parent.
- `DISTINCT ON` + `ORDER BY date_of_lead` keeps the first lead; `DESC` would credit the latest.
- `l.deleted_at` carries its alias, so it can't become ambiguous when more tables join.

Lesson: Cut a many-rows table to one row per key in a CTE before joining it.

20261102-02 Flag which customers traded in October

pandas · **How-to** · isin, np.where, Period

PROBLEM

Needed a Traded / Not Traded column on the filtered customer list for October.

CAUSE

The October BILL rows hold the traders' IDs; each customer just needs checking against that set.

FIX

```
led_req = led[led["ClientID"].isin(cust_req)].copy()
led_req = led_req[(led_req["LedgerType"] == "BILL") &
                  (led_req["CreatedAt"].dt.to_period("M") == pd.Period("2026-10", freq="M"))]

traded_ids = set(led_req["ClientID"].astype(str).str.strip())
customer_filtered = customer_filtered.copy()
customer_filtered["Oct Trading"] = np.where(
    customer_filtered["Client ID"].astype(str).str.strip().isin(traded_ids), "Traded", "Not Traded")
```

WHY IT WORKS

- `isin` gives True/False per customer; `np.where` turns it into labels.
- Strip and cast both sides, so spaces or number-vs-text IDs can't hide a match.
- `.str.lower()` before `isin(['demo', 'suspended'])` also catches `Demo`.

Lesson: A flag column is isin + np.where; clean both sides of the match first.

20261102-03 Total and traded clients per Manager and agent

pandas · **How-to** · groupby, nunique, where, subtotals

PROBLEM

Needed, per Manager and Assigned To, the number of clients and how many traded in October.

CAUSE

A conditional distinct count needs only the traders' IDs in one column.

FIX

```
cf = customer_filtered.copy()
cf["Assigned To"] = cf["Assigned To"].fillna("Unassigned")
cf["Traded_ID"] = cf["Client ID"].where(cf["Oct Trading"].eq("Traded"))

pivot = (cf.groupby(["Manager", "Assigned To"])
           .agg(Total_Clients=("Client ID", "nunique"), Traded_in_Oct=("Traded_ID", "nunique"))
           .reset_index())
pivot["Traded %"] = (pivot["Traded_in_Oct"] / pivot["Total_Clients"] * 100).round(1)
```

WHY IT WORKS

- `.where(cond)` blanks non-traders; `nunique` ignores blanks, so it counts only traders.
- `fillna('Unassigned')` stops groupby dropping clients with no agent.
- Subtotals: group by Manager alone, label `Manager total`, concat, sort with a helper `_order` column.

Lesson: Conditional distinct count = .where() then nunique, no lambda needed.

20261102-04 Calls after conversion would repeat per lead

SQL · **Wrong result** · fan-out, MIN, FILTER, STRING_AGG

PROBLEM

The first calls query joined `call_logs`, `customers`, `customer_trading_profiles` and `leads` together.

CAUSE

A customer with two leads gets every call twice. Grouping only by `zynixx_client_id` would also merge customers with no trading profile.

FIX

```
WITH conv AS (
    SELECT customer_id, MIN(converted_at) AS converted_at
    FROM leads WHERE converted_at IS NOT NULL GROUP BY customer_id
)
SELECT customer_id, zynixx_client_id,
       COUNT(*)                                                    AS calls_after_conversion,
       COUNT(*) FILTER (WHERE since_conversion <= INTERVAL '1 day') AS within_1_day,
       COUNT(answered_at)                                          AS answered_after_conversion,
       STRING_AGG(TO_CHAR(started_at AT TIME ZONE 'Asia/Kolkata', 'DD Mon YY HH24:MI'), ', ' ORDER BY
    started_at) AS call_times
FROM calls_after
GROUP BY customer_id, zynixx_client_id;
```

WHY IT WORKS

- `MIN(converted_at)` per customer leaves one conversion date, so calls can't multiply.
- `COUNT(*)` counts rows; `COUNT(answered_at)` counts only non-blank values, so answered calls.
- Compare stored times (`started_at > converted_at`); convert time zones only for display.

Lesson: Before joining a table, ask: how many rows per key does it have?

20261102-05 Start from the pasted ID list so no ID disappears

SQL · **How-to** · UNNEST, ARRAY, LEFT JOIN, check_status

PROBLEM

With `WHERE zynixx_client_id IN (...)`, IDs with no calls, or that didn't exist, vanished from the result.

CAUSE

The query started from calls; an ID without calls never made it into a row.

FIX

```
WITH id_list AS (
    SELECT DISTINCT UNNEST(ARRAY['S221954', 'P227319', 'A226436']) AS zynixx_client_id
)
SELECT i.zynixx_client_id,
       CASE WHEN i.customer_id IS NULL   THEN 'ID not found'
            WHEN NOT i.in_scope          THEN 'Other portal/project'
            WHEN cv.converted_at IS NULL THEN 'No conversion date'
            ELSE 'OK' END AS check_status,
       COALESCE(a.calls_after_conversion, 0) AS calls_after_conversion
FROM ids i
LEFT JOIN conv cv ON cv.customer_id = i.customer_id
LEFT JOIN agg a   ON a.zynixx_client_id = i.zynixx_client_id;
```

WHY IT WORKS

- `UNNEST(ARRAY[...])` turns the list into rows, in the same format as an IN list.
- LEFT JOINs from the list keep every ID; `check_status` explains each zero.
- Check: rows returned = distinct IDs pasted.

Lesson: When the question is 'for these IDs', start the query from the IDs.

20261102-06 Join a Metabase saved question safely

SQL · **How-to** · {{#3268}}, DISTINCT ON, team list

PROBLEM

Needed agent, TL, manager and team for each call from saved question #3268.

CAUSE

If an agent appears twice in the team list, joining it directly counts each of their calls twice.

FIX

```
team AS (
    SELECT DISTINCT ON (user_id) user_id, agents, tl, manager, team
    FROM {{#3268-team-main}}
    ORDER BY user_id
)
-- check first:
SELECT user_id, COUNT(*) FROM {{#3268-team-main}} GROUP BY user_id HAVING COUNT(*) > 1;
```

WHY IT WORKS

- `{{#3268-...}}` uses a saved question like a table inside a native query.
- `STRING_AGG(DISTINCT agent, ', ' ORDER BY agent)` lists agents once each.
- `(ARRAY_AGG(agent ORDER BY started_at DESC))[1]` is the agent on the latest call.

Lesson: Every lookup table gets a one-row-per-key guard, even a team list.

20261102-07 Keep only the Sales team's calls

SQL · **How-to** · JOIN vs LEFT JOIN, filter on joined table

PROBLEM

The report had to count only calls made by Sales agents.

CAUSE

The team is a column of the joined team list, so the filter belongs on that table.

FIX

```
SELECT team, COUNT(*) AS agents FROM {{#3268-team-main}} GROUP BY team ORDER BY team;  -- exact values

JOIN team t ON t.user_id = cl.agent_user_id
WHERE ... AND t.team = 'Sales'          -- or ILIKE 'sales%' after checking the values
```

WHY IT WORKS

- An inner JOIN plus `t.team = 'Sales'` drops calls by anyone else, including agents not in the list.
- Check the distinct values first, so `ILIKE 'sales%'` doesn't catch `Sales Support`.

Lesson: Look at the real values of a column before filtering on it.

20261102-08 30-day counts beside the day buckets

SQL · **How-to** · FILTER, INTERVAL, cross-check

PROBLEM

Needed calls and answered calls within 30 days of conversion, keeping the existing buckets.

CAUSE

Two more conditional counts in the same aggregation.

FIX

```
COUNT(*)           FILTER (WHERE since_conversion <= INTERVAL '30 days') AS calls_within_30_days,
COUNT(answered_at) FILTER (WHERE since_conversion <= INTERVAL '30 days') AS answered_within_30_days,
```

WHY IT WORKS

- `FILTER` works on any aggregate, including `COUNT(column)`.
- Check: `calls_within_30_days = within_1_day + days_2_to_7 + days_8_to_30` on every row.

Lesson: When two columns must agree, write the check down and run it.

20261102-09 A 30-day request rebuilt three times

Process · **Wrong result** · requirements, output columns

PROBLEM

"Calls within 30 days" became day_0 to day_30 columns, then a stripped query, then a query missing the buckets, before the right version.

CAUSE

It wasn't agreed whether the 30-day columns should replace or be added to the existing ones.

FIX

```
Start from: <the query that's already right>
Add columns: <name> = <definition>, <name> = <definition>
Keep: everything else as it is
One row per: <client / agent / day>
```

WHY IT WORKS

- Naming the exact columns, and what to keep, settles a change in one round.
- "Replace" or "add" is the question to settle before rewriting a working query.

Lesson: Say what to keep, not only what to add.

20261102-10 Admin IDs that share an IP

Metabase · **How-to** · Summarize, distinct count, STRING_AGG, private IPs

PROBLEM

The editor showed how many Admin IDs used each IP, but not which IDs.

CAUSE

Metabase's Summarize has no aggregation that lists values in one cell.

FIX

```
SELECT ip, COUNT(DISTINCT admin_id) AS admin_count,
       STRING_AGG(DISTINCT admin_id::text, ', ' ORDER BY admin_id::text) AS admin_ids
FROM tdx_activity
WHERE ip IS NOT NULL AND ip <> ''
GROUP BY ip
HAVING COUNT(DISTINCT admin_id) > 1
ORDER BY admin_count DESC;
```

WHY IT WORKS

- View the SQL, then Convert this question to SQL, turns the editor question into editable SQL.
- A blank-IP row (10 admins) is missing data, not a shared address.
- `10.x` IPs are private (office, VPN, proxy); IP + user agent is a stronger same-device signal.

Lesson: Know where the editor stops; switch to SQL for lists and complex logic.

Wednesday, 4 November 2026 (covers 3 to 4 Nov)

Focus	Client reports from the ledger (net withdrawals, campaign flag, trading days), a two-sheet broker commission workbook in the MIS design, importing your own modules, and writing functions.
Outcome	11 entries: 2 errors, 3 wrong results and 6 how-tos. The P&L chain became a function at three levels, each checked against the original result.
Review	Repeated: sorting by the key instead of the date before keep='first' (third time). Focus tomorrow: write trading_days(led, start, end) at Level 2 and check it with assert_frame_equal.
Cards	`20261104-01` to `20261104-11`

20261104-01 When a lambda is fine, and when it's 900x slower

pandas · **How-to** · apply, vectorise, benchmark

PROBLEM

Needed a rule for when to use lambdas in pandas.

CAUSE

A lambda inside `.apply` runs once per row in plain Python; one given a whole column or table doesn't.

FIX

```
df.apply(lambda r: r.a + r.b, axis=1)     # 4,718 ms on 500,000 rows
df.a + df.b                               #     5 ms

calls.assign(talk_s=lambda d: d['Talk Time'].dt.total_seconds())   # fine: d is the whole table
max(files, key=lambda f: f.stat().st_mtime)                        # fine: key= function
```

WHY IT WORKS

- Order to try: column operators, np.where/np.select/pd.cut, groupby built-ins, map/str, then apply.
- Surprise on pandas 3: .str.split('|').str[1] was the slowest text method; .str.replace(regex) the fastest.
- Joining text per group (', '.join) has no built-in, so a lambda is right there.

Lesson: Give a lambda whole columns, never one row at a time.

20261104-02 Deposits and withdrawals net of cancellations

pandas · **How-to** · where, groupby, ledger types

PROBLEM

Needed Deposit and Withdraw per client, with WITHDRAW CANCELLED credits subtracted.

CAUSE

The pivot_table used two lambdas that looked rows up again with .loc[x.index].

FIX

```
lt = req_led['LedgerType'].str.strip().str.upper()
cr, dr = req_led['Credit'].fillna(0), req_led['Debit'].fillna(0)
one_pivot = (req_led
    .assign(Deposit=cr.where(lt.eq('DEPOSIT'), 0),
            Withdraw=dr.where(lt.eq('WITHDRAW'), 0) - cr.where(lt.eq('WITHDRAW CANCELLED'), 0))
    .groupby('ClientID', as_index=False)[['Deposit', 'Withdraw']].sum())
```

WHY IT WORKS

- Each row contributes +debit, -credit or 0, so the sum is withdrawals minus cancellations.
- fillna(0) first: one blank Credit would make the row blank and sum would skip it.

> **Lesson:** Build the amount columns, then one plain sum.

20261104-03 np.where got one argument: a bracket closed too late

pandas · **Error** · np.where, &, brackets

PROBLEM

`np.where((df['Balance'] == 0) & (df['Last Trade'] != 'No Trade', 'Yes', 'No'))` failed.

CAUSE

The second condition's `)` closed after 'No', so Python built a tuple and np.where got one argument.

FIX

```
df['Traded Zero'] = np.where(
    (df['Balance'] == 0) & (df['Last Trade'] != 'No Trade'),
    'Yes', 'No')
```

WHY IT WORKS

- Each condition in its own brackets; & | ~ between them; then the true and false values.
- `and` / `or` raise 'truth value of a Series is ambiguous'.
- 0.004 isn't == 0: use .round(2).eq(0) when tiny balances should count.

> **Lesson:** Close every condition's bracket before the first comma.

20261104-04 Latest balance taken from a random row (again)

pandas · **Wrong result** · sort_values, drop_duplicates, repeated

PROBLEM

`balance = req_led.sort_values('ClientID').drop_duplicates('ClientID', keep='first').`

CAUSE

Sorting by the key leaves rows within each client in an undefined order, so 'first' isn't the latest.

FIX

```
balance = (req_led.sort_values('CreatedAt', ascending=False)
                  .drop_duplicates(subset='ClientID', keep='first')
                  [['ClientID', 'CreatedAt', 'Balance']])
```

WHY IT WORKS

- Third time in the journal (29 Oct, 30 Oct, 3 Nov).
- Balance feeds Traded Zero, so one wrong sort spreads into two columns.

> **Lesson:** Before keep='first', the line above must sort by a date.

20261104-05 Campaign flag: 'deposited on' vs 'first deposit was'

pandas · **Wrong result** · first row per group, requirements

PROBLEM

The flag was built for any 1,000 deposit on 25 Aug; the rule was the client's FIRST deposit.

CAUSE

Every deposit was tested instead of only each client's earliest one.

FIX

```
first_dep = dep.sort_values('CreatedAt').drop_duplicates('ClientID', keep='first')
camp_ids = set(first_dep.loc[(first_dep['CreatedAt'].dt.normalize() == pd.Timestamp('2026-08-25'))
                             & (first_dep['Credit'] == 1000), 'ClientID'])
df['Campaign'] = np.where(df['ClientID'].isin(camp_ids), 'Campaign', 'Not Campaign')
```

WHY IT WORKS

- 'First X': sort oldest first, keep the first row per client, then test it.
- 'Only X': also require one deposit in total.
- First within the loaded ledger: check led['CreatedAt'].min().

Lesson: Pin down first / any / only before writing the filter.

20261104-06 Trading days and their spread per client

pandas · **How-to** · nunique, join per group, to_period

PROBLEM

Needed trading days per client, plus the dates and a per-month count as text.

CAUSE

Several bills per day must count once; month labels must stay in calendar order.

FIX

```
bills = (req_led[lt.eq('BILL')].assign(Trade_Date=lambda d: d['CreatedAt'].dt.normalize())
           .drop_duplicates(['ClientID', 'Trade_Date']).sort_values(['ClientID', 'Trade_Date']))
dates = bills.groupby('ClientID')['Trade_Date'].agg(lambda s: ', '.join(s.dt.strftime('%d %b')))
per_month = (bills.assign(M=lambda d: d['Trade_Date'].dt.to_period('M'))
                  .groupby(['ClientID', 'M']).size().reset_index(name='n')
                  .assign(t=lambda d: d['M'].dt.strftime('%b %Y') + ': ' + d['n'].astype(str))
                  .groupby('ClientID')['t'].agg(', '.join))
```

WHY IT WORKS

- drop_duplicates on client + date keeps one row per trading day.
- Sorting by the Period, not the text, keeps months in calendar order.
- Check: days per month add up to Trading Days; 0 days where Last Trade is 'No Trade'.

Lesson: Join text per group with ', '.join after sorting.

20261104-07 Excel export in the MIS design

pandas · **How-to** · openpyxl, ExcelWriter, number_format

PROBLEM

The commission table had to look like the MIS file: coloured bands, gold headers, formats.

CAUSE

pandas writes plain cells; styling needs openpyxl on the sheet pandas just wrote.

FIX

```
with pd.ExcelWriter(path, engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Brokers', index=False, startrow=1)   # row 1 left for bands
    ws = writer.sheets['Brokers']
    ws.merge_cells(start_row=1, start_column=1, end_row=1, end_column=4)
    ws.cell(1, 1).value = 'Admin Panel Broker File'
    for row in ws.iter_rows(min_row=3):
        for cell in row[4:]:
            cell.number_format = '#,##0.00;[Red]-#,##0.00;"-"'
    ws.freeze_panes, ws.auto_filter.ref = 'B3', f"A2:K{ws.max_row}"
```

WHY IT WORKS

- startrow=1 leaves Excel row 1 free; writer.sheets[name] is the openpyxl sheet.
- Number format parts: positive; negative (red); zero (dash).
- One style_sheet() function styles both the Brokers and the Clients sheets.

Lesson: Write with pandas, style with openpyxl, in the same with-block.

20261104-08 NBP always 0: rows filtered away one step early

pandas · **Wrong result** · filter order, isin

PROBLEM

The broker sheet showed 0 NBP for every client.

CAUSE

led_oct_bill kept only BILL rows, so the later filter for BILL + NBP had no NBP rows left.

FIX

```
lt = ['BILL', 'NEGATIVE BALANCE PROTECTION']
led_child = led_oct[led_oct['ClientID'].isin(child)]          # not led_oct_bill
led_child_fil = led_child[led_child['LedgerType'].isin(lt)]
```

WHY IT WORKS

- Check after each filter: df['LedgerType'].value_counts() shows which types survived.
- Same broker rule: .where(is_loss, 'TEXT') keeps numbers numeric; np.where would turn them into text.

Lesson: A filter can't keep rows an earlier filter already removed.

20261104-09 TypeError: insert expected 2 arguments, got 1

Python · **Error** · sys.path, imports, PYTHONPATH

PROBLEM

`sys.path.insert(r'C:\...\migration')` failed while importing ledger_loader from another folder.

CAUSE

list.insert needs a position and an item; there's no default position.

FIX

```
import sys
sys.path.insert(0, r"C:\Users\Administrator\Videos\Office\migration")   # 0 = searched first
from ledger_loader import get_final_ledger
# permanent: setx PYTHONPATH "C:\Users\Administrator\Videos\Office\migration", then restart VS Code
```

WHY IT WORKS

- Import checks sys.modules first, then searches sys.path from position 0; the first match wins.
- append(folder) takes one argument but searches the folder last.
- module.__file__ shows which file was imported; %autoreload 2 picks up edits.

Lesson: insert(0, folder) puts your folder first in the search.

20261104-10 Writing functions in three levels

Python · **How-to** · def, parameters, defaults, None, raise

PROBLEM

Needed to turn the P&L chain into a reusable function, starting as a beginner.

CAUSE

Copying cells between notebooks repeats work and spreads mistakes.

FIX

```
def client_pnl_v1(led):                                          # level 1: wrap working code
    return led.assign(...).groupby(...).sum().assign(...)

def client_pnl_v2(led, start, end, commission_rate=0.05):          # level 2: parameters + default
    rows = led[(led['CreatedAt'] >= start) & (led['CreatedAt'] < end)]
    ...

def client_pnl_v3(led, start=None, end=None, clients=None, commission_rate=0.05):   # level 3
    if missing: raise ValueError(...)
    if clients is not None: rows = rows[rows['ClientID'].isin(clients)]
```

WHY IT WORKS

- def runs nothing; calling the function does. return sends one result out.
- Names inside a function stay inside; only `x = f(...)` outside creates a notebook variable.
- None as a default means 'not given'; raise ValueError stops early with a clear message.

Lesson: Wrap working code first, then turn fixed values into parameters one at a time.

20261104-11 Prove a rewrite didn't change the numbers

pandas · **How-to** · assert_frame_equal, testing

PROBLEM

Needed to confirm each function version matches the existing pnl table.

CAUSE

Looking at a few rows can't prove two tables are identical.

FIX

```
pd.testing.assert_frame_equal(pnl.reset_index(drop=True),
                              client_pnl_v1(led_child_fil).reset_index(drop=True),
                              check_dtype=False)
```

WHY IT WORKS

- Silent when equal; raises an error showing the first difference when not.
- Level 3 is expected to differ: it reads the full ledger, so NBP rows come back in.

Lesson: Every rewrite gets an equality check against the version you trust.

Monday, 9 November 2026

Focus	Metabase call lists for Pull Back and P1: day gaps in Indian time, the latest status per customer, team alignment by the lead's current owner, Metabase variables, and totals with ROLLUP.
Outcome	15 entries: 4 errors, 3 wrong results and 8 how-tos. Both call-list queries run; the rule for picking the current status when two rows tie is still open (20261109-05). Evening: a sign-up page with admin approval and the Ledger explorer page (cards 12 to 15).
Review	Repeated: NULL handling (blank last call; NULLs first in DESC) and a one-to-many join (status history without rn = 1). Focus tomorrow: run the Y19823 diagnostic, agree the tie rule for Lost Lead vs Dormant, then finalise the Pull Back query.
Cards	`20261109-01` to `20261109-15`

20261109-01 Call list: traded in the last 3 days, not called for 2

SQL · **How-to** · INTERVAL, NULL, OR brackets

PROBLEM

Needed the P1 list: last transaction within 3 days and last call more than 2 days ago.

CAUSE

Two traps: a blank `last_call_at` fails every comparison, so never-called clients vanish; and an OR without brackets escapes the portal and project filters.

FIX

```
WHERE ctp.last_txn_date >= CURRENT_TIMESTAMP - INTERVAL '3 days'
  AND (c.last_call_at < CURRENT_TIMESTAMP - INTERVAL '2 days'
       OR c.last_call_at IS NULL)                          -- never called counts too
  AND c.portal_id = {{snippet: portal_id}}
  AND c.project_id = {{snippet: project_id}}
ORDER BY c.last_call_at ASC NULLS FIRST                    -- longest without a call first
```

WHY IT WORKS

- NULL compared with anything is unknown, and WHERE keeps only true rows.
- AND is evaluated before OR: without the brackets, never-called clients from every portal get in.
- WHERE compares instants, so no AT TIME ZONE is needed; INTERVAL '2 days' means 48 hours back.

Lesson: Every filter on a column that can be blank needs a decision about blanks.

20261109-02 interval 3 days fails; CURRENT_DATE AT TIME ZONE skews the gaps

SQL · **Error** · INTERVAL, AT TIME ZONE, ::date, CTE

PROBLEM

`... > interval 3 days` gave a syntax error. The day gaps used `CURRENT_DATE at time zone 'asia/kolkata' - ctp.last_txn_date at time zone 'asia/kolkata'`, and the call column reused the alias Last_Transaction_From_Today.

CAUSE

An interval literal needs quotes. CURRENT_DATE is midnight of the server's date, and AT TIME ZONE returns different types for a date and a timestamp, so the subtraction converts one with the server's zone and is off by hours.

FIX

```
WITH base AS (
    SELECT ...,
           (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date
             - (ctp.last_txn_date AT TIME ZONE 'Asia/Kolkata')::date AS days_since_last_transaction,
           (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date
             - (c.last_call_at AT TIME ZONE 'Asia/Kolkata')::date    AS days_since_last_call
    FROM ...
)
SELECT * FROM base
WHERE days_since_last_transaction > 3
  AND (days_since_last_call > 4 OR days_since_last_call IS NULL)
```

WHY IT WORKS

- Correct literal: `INTERVAL '3 days'`.
- `(ts AT TIME ZONE 'Asia/Kolkata')::date` is the calendar date in India; date minus date is whole days.
- A column alias can't be used in the same SELECT's WHERE, so compute in a CTE and filter outside it.
- The shown day counts and the filter now use the same numbers.

Lesson: Make both sides the same type, on the same clock, before subtracting dates.

20261109-03 ROW_NUMBER written wrong, and every status row joined

SQL · **Error** · ROW_NUMBER, OVER, one-to-many

PROBLEM

`row_number() over partition by(customer_id order by new_last_txn_date desc)` in a joined subquery failed, joined on `cu.id`, and a `select * from customer_status_history` followed the query.

CAUSE

The brackets belong after OVER. Even with that fixed, nothing kept only row 1, so each client would repeat once per history row; and `cu.id` is the current-user model's own ID, not the customer.

FIX

```
WITH ranked_status AS (
    SELECT customer_id, new_status, occurred_at,
           ROW_NUMBER() OVER (PARTITION BY customer_id
                              ORDER BY occurred_at DESC NULLS LAST, id DESC) AS rn
    FROM customer_status_history
),
latest_status AS (
    SELECT customer_id, new_status FROM ranked_status WHERE rn = 1
)
...
LEFT JOIN latest_status ls ON ls.customer_id = c.id
```

WHY IT WORKS

- Shape: `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...) AS rn`.
- A window function can't be filtered in the SELECT that computes it: number first, filter rn = 1 outside.
- Join on the same customer key the other joins use (c.id).
- A second statement in a Metabase question is an error: test queries go in their own question.

Lesson: A 'latest row' join without rn = 1 is a one-to-many join.

20261109-04 Latest status picked a row with no date: DESC puts NULLs first

SQL · **Wrong result** · ORDER BY, NULLS LAST, tie-breaker

PROBLEM

The current status didn't match the CRM for some clients when sorting the history by date DESC.

CAUSE

In PostgreSQL, DESC puts NULL values first. History rows with no date ranked as the newest.

FIX

```
ORDER BY customer_id,
         occurred_at DESC NULLS LAST,     -- blank dates go to the bottom
         id DESC                          -- tie-breaker: the last row inserted

-- show which row was picked, so the result can be checked by eye
ls.status_changed_at AT TIME ZONE 'Asia/Kolkata' AS status_changed_at
```

WHY IT WORKS

- Defaults: ASC puts NULLs last, DESC puts NULLs first. NULLS LAST / NULLS FIRST overrides.
- A tie-breaker makes the pick repeatable; without one, ties are returned in any order.
- Showing the sort column next to the result turns 'looks wrong' into something you can check.

Lesson: Every DESC on a column that can be blank needs NULLS LAST, or a deliberate NULLS FIRST.

20261109-05 Lost Lead shown as Dormant: several rows tie on the sort column (open)

SQL · **Wrong result** · ties, LIMIT 1, business rule

PROBLEM

Y19823 shows Dormant. `ORDER BY new_last_txn_date DESC LIMIT 1` on its history shows Lost Lead (previous Dormant, both txn dates 23 Sep 2025 4:12 PM).

CAUSE

The status changed without a new transaction, so several rows share the same new_last_txn_date. LIMIT 1 returns whichever tied row it reads first; the query broke the tie by occurred_at and picked Dormant. Status: open until the diagnostic is run.

FIX

```
-- 1. See every tied row and the rank the query gives it
SELECT h.customer_id, h.id, h.previous_status, h.new_status, h.new_last_txn_date, h.occurred_at,
       ROW_NUMBER() OVER (PARTITION BY h.customer_id
                          ORDER BY h.new_last_txn_date DESC, h.occurred_at DESC NULLS LAST, h.id DESC)
    AS rn
FROM customer_trading_profiles ctp
JOIN customer_status_history h ON h.customer_id = ctp.customer_id
WHERE ctp.zynixx_client_id = 'Y19823'
ORDER BY rn

-- 2. If the rule is "Lost Lead stays until the client trades again":
ORDER BY new_last_txn_date DESC, (new_status = 'Lost Lead') DESC, occurred_at DESC NULLS LAST, id DESC
```

WHY IT WORKS

- previous_last_txn_date = new_last_txn_date means no new transaction: a tie is guaranteed.
- A check that agrees by luck (LIMIT 1 on a tied column) proves nothing.
- Which status wins a tie is a business rule, not a SQL question: agree it, then encode it.

Lesson: Before trusting ORDER BY x LIMIT 1, check that x is unique within the group.

20261109-06 Team alignment from the lead's current owner, not the last caller

SQL · **How-to** · LEFT JOIN, models, name join

PROBLEM

P1 took TL, manager and team from `c.last_call_agent_id`; they should follow the lead's owner.

CAUSE

Two different people: the last caller and the current owner. The owner comes from the current-user model, then the team list by name.

FIX

```
LEFT JOIN {{#3367-current-user}} cu ON c.id = cu.customer_id          -- who owns the lead now
LEFT JOIN {{#3268-team-main}} tm    ON cu.current_owner = tm.agents   -- that owner's TL, manager, team

-- owners that don't match the team list (spelling, extra spaces)
SELECT DISTINCT cu.current_owner
FROM {{#3367-current-user}} cu
LEFT JOIN {{#3268-team-main}} tm ON cu.current_owner = tm.agents
WHERE tm.agents IS NULL
```

WHY IT WORKS

- LEFT JOINs keep clients whose owner isn't in the team list (blank TL) instead of dropping them.
- A WHERE on tm.team (e.g. 'Pull Back') turns the join into an inner join on purpose.
- Name joins are fragile: LOWER(TRIM(a)) = LOWER(TRIM(b)) if the check finds near-matches.

Lesson: Decide which person a row belongs to before joining the team list.

20261109-07 Remove clients who transacted today (Indian time)

SQL · **How-to** · ::date, AT TIME ZONE, calendar day

PROBLEM

The P1 list had to drop clients whose last transaction is today.

CAUSE

`last_txn_date` is the latest transaction, so 'before today' means no transaction today. 'Today' must be the Indian calendar day, not the server's day or the last 24 hours.

FIX

```
AND (ctp.last_txn_date AT TIME ZONE 'Asia/Kolkata')::date
      < (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date        -- not today (IST)

-- fully calendar-based alternative: 1 to 3 days ago, never today
AND (ctp.last_txn_date AT TIME ZONE 'Asia/Kolkata')::date
      BETWEEN (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date - 3
          AND (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date - 1
```

WHY IT WORKS

- A transaction at 00:30 IST today is excluded; one at 23:50 IST yesterday stays.
- last_txn_type includes deposits, so this removes anyone who transacted, not only traded.
- Mixing a rolling 72-hour window with a calendar 'today' is easy to misread: pick one.

Lesson: Say whether a window is rolling hours or calendar days, and in which time zone.

20261109-08 syntax error at or near "$1": IN without brackets

Metabase · **Error** · variables, IN, [[ ]]

PROBLEM

ERROR: syntax error at or near "$1" Position: 2824, after adding `and tm.tl in {{tl}}` and `and cu.current_owner in {{current_owner}}`.

CAUSE

Metabase replaces each variable with a placeholder ($1, $2...), so the SQL became `IN $1`. IN needs its list in brackets.

FIX

```
  [[AND tm.tl IN ({{tl}})]]
  [[AND cu.current_owner IN ({{current_owner}})]]
-- variable type: Text, "Allow multiple values" on, "Always require a value" off
```

WHY IT WORKS

- Write the variable exactly where a literal value would go, brackets included.
- [[...]] drops the whole line when the box is empty; keep the AND inside the brackets.
- Field Filter variables need real table columns; with model joins, use Text variables.

Lesson: When Metabase reports $1, look at the line where the first variable sits.

20261109-09 Multiple ResultSets were returned by the query

Metabase · **Error** · semicolon, statements

PROBLEM

Multiple ResultSets were returned by the query.

CAUSE

The query ended with ; followed by a comment. The driver splits at the semicolon, so the comment became a second, empty statement. A leftover test SELECT below the query does the same.

FIX

```
ORDER BY c.last_call_at ASC NULLS FIRST          -- longest without a call first
-- no semicolon, nothing after the query
```

WHY IT WORKS

- A Metabase question runs exactly one statement.
- Ctrl+End in the editor shows anything hiding below the query.

Lesson: One statement per Metabase question, and no trailing semicolon.

20261109-10 A Total row with ROLLUP, at the bottom or the top

SQL · **How-to** · ROLLUP, GROUPING, totals

PROBLEM

Needed counts per TL and per owner from the P1 model, with a Total row.

CAUSE

ROLLUP adds the total row; GROUPING() tells it apart from a group whose TL is blank.

FIX

```
SELECT CASE WHEN GROUPING(tl) = 1 THEN 'Total'
            ELSE COALESCE(tl, 'No TL') END AS tl,
       COUNT(*) AS clients
FROM {{#6437-p1}}
GROUP BY ROLLUP (tl)
ORDER BY GROUPING(tl), clients DESC          -- Total last;  GROUPING(tl) DESC puts it first

-- per TL and owner, with a subtotal per TL
GROUP BY ROLLUP (tl, current_owner)
ORDER BY GROUPING(tl) DESC, tl, GROUPING(current_owner) DESC, clients DESC
```

WHY IT WORKS

- GROUPING(col) is 1 on the rolled-up row and 0 on normal rows.
- COALESCE(tl, 'Total') would also label real clients with no TL as Total.
- COUNT(DISTINCT zynixx_client_id) if the model can repeat a client.

> **Lesson:** Label totals with GROUPING(), never by testing for NULL.

20261109-11 DISTINCT ON and ROW_NUMBER give the same latest row

SQL · **How-to** · DISTINCT ON, ROW_NUMBER, IS DISTINCT FROM

PROBLEM

Wanted to know whether to use DISTINCT ON or ROW_NUMBER for the latest status.

CAUSE

With the same ORDER BY they pick the same row. ROW_NUMBER also does top-N (rn <= 3) and works in other databases.

FIX

```
WITH a AS (SELECT DISTINCT ON (customer_id) customer_id, new_status
           FROM customer_status_history
           ORDER BY customer_id, occurred_at DESC NULLS LAST, id DESC),
b AS (SELECT customer_id, new_status
      FROM (SELECT customer_id, new_status,
                   ROW_NUMBER() OVER (PARTITION BY customer_id
                                      ORDER BY occurred_at DESC NULLS LAST, id DESC) AS rn
            FROM customer_status_history) h
      WHERE rn = 1)
SELECT COUNT(*) AS customers_that_differ          -- should be 0
FROM a FULL JOIN b USING (customer_id)
WHERE a.new_status IS DISTINCT FROM b.new_status
```

WHY IT WORKS

- Changing the method doesn't change the answer; only the ORDER BY decides which row wins.
- IS DISTINCT FROM treats two blanks as equal; <> would miss differences involving NULL.
- FULL JOIN also catches a customer present in only one version.

> **Lesson:** When two ways of writing a rule should agree, prove it with a query that counts differences.

20261109-12 A sign-up page: built-in form, extra fields, admin approval

Django · **How-to** · UserCreationForm, clean_email, is_active

PROBLEM

The reports site had login but no way to create an account.

CAUSE

Django ships login and logout (django.contrib.auth.urls) but no sign-up view. It does ship the form: UserCreationForm (username, password, password again).

FIX

```
class SignUpForm(UserCreationForm):
    first_name = forms.CharField(max_length=150)
    last_name = forms.CharField(max_length=150)
    email = forms.EmailField()

    class Meta(UserCreationForm.Meta):
        model = User
        fields = ("username", "first_name", "last_name", "email")

    def clean_email(self):
        email = self.cleaned_data["email"].strip().lower()
        if User.objects.filter(email__iexact=email).exists():
            raise forms.ValidationError("An account with this email already exists.")
        return email

# view: user = form.save(commit=False); user.is_active = False; user.save()   (when
    SIGNUP_NEEDS_APPROVAL)
```

WHY IT WORKS

- Extending UserCreationForm keeps Django's password checks and hashing for free.
- clean_<field> runs after the basic checks; email__iexact catches the same email in different case.
- is_active = False is Django's own on/off switch: an admin ticks Active in /admin/ to approve.

Lesson: On an internal-data site, open sign-up means anyone on the network sees the data: add approval.

20261109-13 Every form error showed twice

Django · **Wrong result** · templates, non_field_errors, as_p

PROBLEM

A wrong login showed 'Please enter a correct username and password' twice; the report forms did the same for their top-of-form errors. All tests passed.

CAUSE

The templates printed {{ form.non_field_errors }} and then {{ form.as_p }}, and as_p already prints the non-field errors at the top.

FIX

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}          {# already includes the non-field errors: no separate line #}
  <button class="btn" type="submit">Log in</button>
</form>
```

WHY IT WORKS

- as_p, as_div and as_table all render non-field errors first.
- Tests that use assertContains pass whether a message appears once or twice.
- Found by clicking through in a browser, not by the tests.

Lesson: After the tests pass, click through the page once: some bugs are only visible.

20261109-14 An unapproved user only sees 'incorrect username or password'

Django · **How-to** · is_active, AuthenticationForm

PROBLEM

A new user waiting for approval got the normal 'correct username and password' error at login.

CAUSE

Django's login backend treats inactive users as unknown on purpose, so the page never reveals whether an account exists.

FIX

```
<!-- registration/login.html, under the form -->
<p class="muted">No account yet? <a href="{% url 'core:signup' %}">Sign up</a></p>
<p class="muted" style="font-size:12px">New accounts can log in once an admin approves them.</p>

<!-- and the sign-up view says it at the moment of signing up -->
messages.success(request, "Account created. An admin needs to approve it before you can log in.")
```

WHY IT WORKS

- Saying 'this account is inactive' would let anyone test which usernames exist.
- So the explanation goes where the user already is: the sign-up message and a note on the login page.

Lesson: When the system can't say why, tell the user beforehand.

20261109-15 A DataFrame as a sortable, paged HTML table (Ledger explorer)

Django · **How-to** · to_dict('records'), Paginator, request.GET

PROBLEM

Needed ledger rows on a page: filter by month, client, type; sort by clicking a heading; 50 rows a page; CSV download.

CAUSE

Templates read dict keys with dots, so column names with spaces or capitals don't work; and every link must keep the current filters.

FIX

```
table = led.rename(columns={v: k for k, v in COLUMNS.items()})[list(COLUMNS)]   # short names
key = sort.lstrip("-") if sort.lstrip("-") in COLUMNS else "created"               # whitelist
table = table.sort_values(key, ascending=not sort.startswith("-"), kind="stable")
page_obj = Paginator(table.to_dict("records"), 50).get_page(request.GET.get("page"))

def url_with(request, **changes):          # current address with some values changed
    params = request.GET.copy()
    for k, v in changes.items():
        params.pop(k, None) if v is None else params.__setitem__(k, v)
    return "?" + params.urlencode()
```

WHY IT WORKS

- to_dict('records') gives one dict per row: exactly what {% for row in page_obj %} needs.
- Filters use method='get', so a filtered page can be bookmarked and shared.
- get_page() turns a bad page number into the first or last page instead of an error.

Lesson: Keep numbers as numbers in Python; format them in the template (|floatformat:'2g').

Tuesday, 10 November 2026

Focus	Broker referral program report: children per referrer by status, October activity, program targets (required active clients, new clients, trade value with 2% growth), monthly trade value columns and the status and conversion lookups.
Outcome	11 entries: 1 error, 4 wrong results and 6 how-tos. The program report runs end to end with targets, eligibility and a reason per broker; monthly trade values reconcile with the ledger.
Review	Repeated: business rules whose edge cases weren't tried (no active clients; just below the minimum) and one-to-many merges. Open: should '6 children' count demo children, and which 30-day window does the program mean? Focus tomorrow: confirm both, then export the program sheet.
Cards	`20261110-01` to `20261110-11`

`20261110-01`

Every child in exactly one group: np.select with a priority

pandas · **How-to** · np.select, named agg, is_demo as text

PROBLEM

Per referrer: Overall, Converted (first trade date), Not converted (no trade, not demo) and Demo, where the three groups must add up to Overall.

CAUSE

A demo account can also have a trade date, so 'traded' and 'demo' overlap. Groups only add up if each child gets one label, decided in a fixed order.

FIX

```
kids = bf.drop_duplicates('zynixx_client_id').copy()
is_demo = kids['is_demo'].astype(str).str.strip().str.lower().isin(['true', 'yes', '1'])
traded = kids['first_trade_date'].notna()
kids['status'] = np.select([is_demo, traded], ['Demo', 'Converted'], default='Not converted')

summary = (kids.groupby('referrer_client_id')
               .agg(Parent=('referrer_name', 'first'), Overall=('zynixx_client_id', 'size'),
                    Converted=('status', lambda s: s.eq('Converted').sum()),
                    **{'Not converted': ('status', lambda s: s.eq('Not converted').sum())},
                    Demo=('status', lambda s: s.eq('Demo').sum())))
```

WHY IT WORKS

- np.select takes the first true condition, so Demo is decided before Converted.
- Reading is_demo as text handles True/False and 'True'/'False' (Metabase exports).
- **{...} passes output names that contain spaces.

Lesson: When categories must add up to a total, give each row one label, in a stated order.

20261110-02 isin(is_demo) compared client IDs with True and False

pandas · **Wrong result** · isin, boolean mask

PROBLEM

`non_demo = bf.loc[~bf['zynixx_client_id'].isin(is_demo), 'zynixx_client_id']` kept every client, demo accounts included.

CAUSE

is_demo is a True/False mask, not a list of IDs. No ID equals True or False, so isin() was False everywhere and ~ made it True everywhere.

FIX

```
# a mask selects rows directly
real_ids = set(kids.loc[~kids['is_demo'], 'zynixx_client_id'])

# check: real + demo must equal all children
print(len(real_ids), kids['is_demo'].sum(), len(kids))
```

WHY IT WORKS

- isin() wants values to look for; a mask is used with .loc[mask] or df[mask].
- The bug raised no error: only a count would have shown it.

Lesson: After every filter, print how many rows it kept.

20261110-03 SettingWithCopyWarning when adding a Month column to a filtered table

pandas · **Error** · SettingWithCopyWarning, Period, date filter

PROBLEM

`led_bf_req['Month'] = pd.to_datetime(...).dt.to_period('M')` on a filtered table gave a SettingWithCopyWarning, followed by `led_bf_req[led_bf_req['Month'] == '2026-10']`.

CAUSE

The table was a slice of another table; pandas can't tell whether the new column should reach the original. A Month column was only needed to filter, which a date range does directly.

FIX

```
month_bills = bills[(bills['CreatedAt'] >= m_start) & (bills['CreatedAt'] < m_end)]   # no new column

# when a filtered table really needs a new column, copy it first
part = bills[bills['ClientID'].isin(ids)].copy()
part['Month'] = part['CreatedAt'].dt.to_period('M')
```

WHY IT WORKS

- A half-open date range filters without changing the table.
- .copy() makes the slice its own table, so assigning a column is unambiguous.

Lesson: Filter with conditions; add columns only to tables you own (.copy()).

20261110-04 Broker program report: yes/no flags per child, summed per broker

pandas · **How-to** · flags, isin, groupby sum

PROBLEM

Per broker: Children, Demo Children, Added This Month, Active This Month, Active within 30 days, 6-child eligibility and Trade Value.

CAUSE

Each measure is a question about one child (added? active?). Answer it per child as True/False, then sum per broker.

FIX

```
kids['added_this_month'] = (kids['created_at'] >= m_start) & (kids['created_at'] < m_end)
kids['active_this_month'] = kids['zynixx_client_id'].isin(month_bills['ClientID'])
kids['active_30_days'] = kids['zynixx_client_id'].isin(recent_bills['ClientID'])
kids['trade_value'] = kids['zynixx_client_id'].map(month_bills.groupby('ClientID')['Debit'].sum()).
    fillna(0)

report = (kids.groupby('referrer_client_id')
              .agg(Children=('zynixx_client_id', 'size'),
                   **{'Demo Children': ('is_demo', 'sum'), 'Active This Month': ('active_this_month',
    'sum'),
                      'Trade Value': ('trade_value', 'sum')}))
```

WHY IT WORKS

- Summing a True/False column counts the Trues.
- isin() answers 'did this child appear in those BILL rows?' without a merge.
- Checks: real children found in the ledger, and report total = ledger total.

Lesson: Turn each measure into a per-row yes/no first; the summary is then just sums.

20261110-05 Program targets: required active, new clients still to add, minimum trade value

pandas · **How-to** · clip, np.select, settings

PROBLEM

Required active = 2 x active this month; at least 6 new children (show how many are still needed); minimum trade value = required active x 10,000; eligibility with a reason.

CAUSE

Targets are formulas on the report's columns; 'still needed' must never go below zero.

FIX

```
ACTIVE_MULTIPLIER, NEW_CLIENTS_TARGET, TRADE_PER_ACTIVE = 2, 6, 10_000
prog['Required Active Clients'] = (prog['Active This Month'] * ACTIVE_MULTIPLIER).clip(
    lower=REQUIRED_ACTIVE_FLOOR)
prog['Add New Clients'] = (NEW_CLIENTS_TARGET - prog['Added This Month']).clip(lower=0)
prog['Minimum Trade Value'] = (prog['Required Active Clients'] * TRADE_PER_ACTIVE).clip(
    lower=MIN_TRADE_VALUE)
prog['Reason'] = np.select([new_ok & trade_ok, ~new_ok & ~trade_ok, ~new_ok],
                           ['', 'Needs new clients and trade value', 'Needs new clients'],
                           default='Needs trade value')
```

WHY IT WORKS

- .clip(lower=0) stops 'still needed' going negative for brokers who added more than 6.
- Rules as named settings at the top: changing the program means changing one number.
- np.select gives each broker exactly one reason, checked in order.

Lesson: Put program rules in named settings, not inside formulas.

20261110-06 A broker with no active clients was eligible without trading

pandas · **Wrong result** · edge case, floor

PROBLEM

A broker with 0 active clients, 0 trade value and 7 new clients came out ELIGIBLE.

CAUSE

2 x 0 active = 0 required, so the minimum trade value was 0, and 0 >= 0 passes.

FIX

```
REQUIRED_ACTIVE_FLOOR = 6        # every broker needs at least 6 active clients
MIN_TRADE_VALUE = 60_000         # and at least 60,000 trade value
prog['Required Active Clients'] = (prog['Active This Month'] * 2).clip(lower=REQUIRED_ACTIVE_FLOOR)
prog['Minimum Trade Value'] = (prog['Required Active Clients'] * 10_000).clip(lower=MIN_TRADE_VALUE)
```

WHY IT WORKS

- Any target built by multiplying is zero when the input is zero.
- A floor gives every broker a real minimum, whatever their history.
- Found by testing a made-up broker with zero activity, not by the real rows.

Lesson: Test every business rule with a zero row and a just-below row.

20261110-07 max() set the wrong target for a broker just below the minimum

pandas · **Wrong result** · np.where, either/or rule

PROBLEM

Rule: below the minimum -> target is the minimum; minimum met -> target is trade value + 2%. With max(minimum, trade x 1.02), a broker at 59,500 against 60,000 got 60,690.

CAUSE

max() is 'whichever is higher', not 'if met then growth else minimum'. They differ just below the minimum, where 2% growth already passes it.

FIX

```
trade_ok = prog['Trade Value'] >= prog['Minimum Trade Value']
prog['Target Trade Value'] = np.where(trade_ok,
                                      (prog['Trade Value'] * 1.02).round(2),   # already met: +2%
                                      prog['Minimum Trade Value'])              # not met: the minimum
prog['Target Basis'] = np.where(trade_ok, "+2% growth", "Minimum")
```

WHY IT WORKS

- np.where(condition, if_true, if_false) is a literal if/else per row.
- trade_ok is used for both the target and eligibility, so they can't disagree.
- Target Basis shows which rule applied, so the table explains itself.

Lesson: Write the rule exactly as stated (if/else) before reaching for a shortcut like max().

20261110-08 Trade value as one column per month

pandas · **How-to** · to_period, pivot_table, reindex

PROBLEM

Needed September and October trade value as separate columns, with Added and Active still counting October only.

CAUSE

Two windows: the report month and the trade months. One start/end can't serve both.

FIX

```
REPORT_MONTH = pd.Period('2026-10', freq='M')
TRADE_MONTHS = pd.period_range('2026-09', '2026-10', freq='M')

tv = (trade_bills.assign(Month=trade_bills['CreatedAt'].dt.to_period('M'))
                 .merge(kids[['zynixx_client_id', 'referrer_client_id']], left_on='ClientID',
                        right_on='zynixx_client_id')
                 .pivot_table(index='referrer_client_id', columns='Month', values='Debit',
                              aggfunc='sum', fill_value=0)
                 .reindex(columns=TRADE_MONTHS, fill_value=0))          # every month gets a column
tv.columns = [f"Trade Value {m.strftime('%b %Y')}" for m in tv.columns]
```

WHY IT WORKS

- to_period('M') labels each row with its month; pivot_table turns months into columns.
- reindex keeps a column for a month with no trades, so the layout never changes.
- Period arithmetic: REPORT_MONTH + 1 and - 1 give the next and previous months.

Lesson: Name each time window separately in the settings; never reuse one start/end for two meanings.

20261110-09 Lookup merges could duplicate brokers; is_broker listed twice

pandas · **Wrong result** · merge, validate, duplicate columns

PROBLEM

report_s merged report with bf_status and af by broker ID; the column list named is_broker twice, and `report_s.columns` ran before report_s existed.

CAUSE

A lookup file with an ID twice turns one broker into two rows; selecting a name twice creates two columns with the same name.

FIX

```
status = (bf_status.assign(zynixx_client_id=bf_status['zynixx_client_id'].astype(str).str.strip())
                   .drop_duplicates('zynixx_client_id')[['zynixx_client_id', 'is_broker']])
report_s = report.merge(status, left_on='Broker', right_on='zynixx_client_id',
                        how='left', validate='many_to_one')
print("Brokers:", len(report), "->", len(report_s), "(must be equal)")
```

WHY IT WORKS

- drop_duplicates on the lookup makes it one row per ID.
- validate='many_to_one' raises an error if that ever stops being true.
- Cells run out of order hide NameErrors: Restart and Run All before trusting a result.

Lesson: Guard every lookup to one row per key, and compare row counts before and after.

20261110-10 What 'Active within 30 days' measures

pandas · **How-to** · rolling window, as_of

PROBLEM

Asked how the 30-day number was calculated, since it differs from Active This Month.

CAUSE

It counts back 30 x 24 hours from the latest ledger entry (as_of), so it covers late October plus early November, and moves every time the ledger is refreshed.

FIX

```
as_of = led['CreatedAt'].max()                                  # window end: latest ledger entry
recent_bills = bills[bills['CreatedAt'] > as_of - pd.Timedelta(days=30)]
print("30-day window:", as_of - pd.Timedelta(days=30), "to", as_of)

# alternatives:  as_of = m_end  (30 days up to the end of the report month, fixed once the month ends)
#                as_of = pd.Timestamp.now()  (30 days up to today)
```

WHY IT WORKS

- A child active only on 1-10 Oct counts in October but not in this window, and vice versa for November.
- Printing the window makes the definition visible in every run.

Lesson: A rolling window needs its end date printed, or the number can't be reproduced.

20261110-11 Active previous month, and where each new line goes

pandas · **How-to** · Period arithmetic, dependency order

PROBLEM

Added 'Active Previous Month' (September) beside 'Active This Month' (October), and needed to know where to insert each line.

CAUSE

Each line can only use what's defined above it: dates, then BILL slice, then per-child flag, then the count, then anything that combines counts.

FIX

```
p_start, p_end = (REPORT_MONTH - 1).start_time, REPORT_MONTH.start_time     # next to m_start, m_end
prev_bills = bills[(bills['CreatedAt'] >= p_start) & (bills['CreatedAt'] < p_end)]
kids['active_prev_month'] = kids['zynixx_client_id'].isin(prev_bills['ClientID'])
# in .agg:  'Active Previous Month': ('active_prev_month', 'sum'),
report['Active Change'] = report['Active This Month'] - report['Active Previous Month']   # after
    report
```

WHY IT WORKS

- REPORT_MONTH - 1 follows the setting: change the report month and 'previous' follows.
- p_end = the report month's start, so no trade is counted in both months.
- Out-of-order lines raise NameError or KeyError naming the missing piece.

Lesson: Put each new line next to its sibling at the same stage of the pipeline.

Wednesday, 11 November 2026

Focus	The Thailand Plan for management (its rules against the spreadsheet), JSON text in Excel, and the lead-source report: all-time deposits, conversions by event date, source labels, October and November side by side, and the export.
Outcome	15 entries: 3 errors, 6 wrong results and 6 how-tos. The Thailand Plan is drafted with seven decisions for management; the lead report runs for any month and exports a sheet per month. Evening: Day 1 of the notes on using an LLM through its API (card 15).
Review	Repeated: definitions that differed without anyone noticing (demo children, 30-day vs month, source labels, which date a column counts by) and a check that compared unlike row sets. Open: the SLNo. check on the 113,534 repeated ledger rows, and management's answers on the Thailand decisions.
Cards	`20261111-01` to `20261111-15`

`20261111-01`

LET: work a value out once, show a message when it's below 6

Excel · **How-to** · LET, IF, text joining

PROBLEM

=MAX(INT(D2/2),INT(J2+(J2*0.1))) had to print 'Currently ineligible: add N more clients to qualify' whenever the result was below 6.

CAUSE

Without LET the MAX(...) has to be written three times (test, message, result), and the copies drift apart when one is edited.

FIX

```
=LET(v, MAX(INT(D2/2), INT(J2*1.1)),
     need, 6 - v,
     IF(v < 6,
        "Currently ineligible: add " & need & IF(need = 1, " more client", " more clients") & " to
    qualify",
        v))
```

WHY IT WORKS

- LET names a value once (v, need) and reuses it, like a variable.
- & joins text and numbers into one message.
- IF(need = 1, ...) keeps the grammar right for one client.

Lesson: When a formula repeats a calculation, name it with LET.

20261111-02 A message in a number column turned the target into #VALUE!

Excel · **Wrong result** · #VALUE!, mixed column, IFERROR

PROBLEM

In thailand_tripv3, Expected Trade Value =(S2*1.05)*Q2 showed #VALUE! wherever Q held the 'Currently ineligible' message.

CAUSE

Q mixed numbers and text. Any formula that multiplies Q fails on the text rows, and SUM skips them silently.

FIX

```
Q2  Required active:   =MAX(6, INT((D2-E2)/2), ROUNDUP(ROUND(I2*1.1, 6), 0))     (always a number)
R2  Status:            =IF(AND(D2-E2>=6, I2>=6), "Entered", "Not entered: needs " & MAX(0, 6-I2) & "
    more active")
T2  Trade value target: =IF(I2=0, 0, L2/I2*1.05*Q2)
```

WHY IT WORKS

- One column, one type: numbers that feed other formulas never share a column with messages.
- IF(I2=0, 0, ...) also stops a #DIV/0! for brokers with no active clients (624 in October).
- ROUND inside ROUNDUP stops 11.000000000000002 rounding up to 12.

Lesson: Keep the number and the message in separate columns.

20261111-03 The plan's rules and the spreadsheet's formulas said different things

Excel · **Wrong result** · rule definitions, INT, demo children

PROBLEM

Drafting the Thailand Plan from thailand_tripv3: the written rules and the formulas disagreed in five places.

CAUSE

Each formula was written on its own: children counted demo accounts, growth used the 30-day count not October's, 'Half Clients Active' used Converted / 2, INT dropped growth below 10 active, and nothing stopped a requirement below 6.

FIX

```
rule as written          sheet as built                       effect (Oct 2026)
50% of children          INT(Children/2), demo included       T66541: 80, not 49
10% activity growth      INT(Active_30days*1.1)               7 x 1.1 = 7.7 -> 7
6 active minimum         requirement can be 4 or 5            3 entrants: #VALUE!
(third base for 50%)     Half Clients Active = Converted/2    3 definitions of 50%
```

WHY IT WORKS

- INT always rounds down, so 10% of anything under 10 becomes 0.
- A floor of 6 in the formula makes rule 3 consistent with rule 2.
- Cleaning up all four cut the requirement from 421 to 343 active clients and the target from 42.5M to 31.0M.

Lesson: Write each rule as one formula with named inputs before building columns from it.

20261111-04 Pull Client_Id out of JSON text in a cell

Excel · **How-to** · TEXTAFTER, TEXTBEFORE, REGEXEXTRACT, Flash Fill

PROBLEM

A column held JSON like {"Client_Name":...,"Client_Id":"P157544",...}; only the client ID was needed.

CAUSE

The ID sits between known markers: after "Client_Id":" and before the next quote.

FIX

```
=IFERROR(TEXTBEFORE(TEXTAFTER(AZ2, "Client_Id"":"""), """"), "")      Excel 365 / 2021
=IFERROR(REGEXEXTRACT(AZ2, "Client_Id\W+(\w+)", 2), "")                  Excel 365 (2024 updates)
Ctrl+E (Flash Fill) after typing the first ID by hand                     one-off, not live
Power Query: Transform > Parse > JSON                                      every field, monthly
```

WHY IT WORKS

- Inside an Excel string, """" is one quote character: quotes are doubled.
- \W+ skips the punctuation after the key, (\w+) captures the ID, return mode 2 returns the capture.
- Flash Fill guesses a pattern and writes plain values: check a few rows.

Lesson: Extract between markers, not by counting characters.

20261111-05 113,534 repeated ledger rows in every month (all-time deposits too high?)

pandas · **Wrong result** · duplicates, SLNo., ledger overlap

PROBLEM

All-time deposits looked wrong. Checks: 0 clients under 2 leads, IDs all matched, only one DEPOSIT type, nobody deposited before their lead, but 113,534 rows repeated on client, time, type and amount, spread over every month since Sep 2025.

CAUSE

Not yet confirmed. A spread like that isn't a single base/append overlap: either an append file repeats history, two appends overlap, or the rows are separate real entries that look alike.

FIX

```
KEY = ['ClientID', 'CreatedAt', 'LedgerType', 'Debit', 'Credit']
led = get_final_ledger(as_pandas=True, columns=tuple(KEY + ['SLNo.']))
before = len(led)
led = led.drop_duplicates(subset=KEY + ['SLNo.'])        # same values AND same ledger ID = a copy
print("True copies removed:", before - len(led))
```

WHY IT WORKS

- Dropping on the values alone would also delete real trades that close in the same second for the same amount.
- Adding the ledger's own row ID removes only true copies.
- Status: open until the SLNo. counts per source (base, appends, across) are read.

Lesson: Before deleting duplicates, prove they are copies with the record's own ID.

20261111-06 A check compared two different sets of rows

pandas · **Wrong result** · diagnostics, like with like

PROBLEM

'converted_at filled - raw: 35,244 | after parsing: 31,192' suggested 4,052 dates lost in parsing.

CAUSE

The raw count was taken on the whole file; the parsed count on df, which had the Old Portal_1YT rows removed. The gap may be those rows, not parsing.

FIX

```
raw_f = raw[raw['tag'] != 'Old Portal_1YT']              # same filter as df
parsed = pd.to_datetime(raw_f['converted_at'], errors='coerce')
lost = raw_f['converted_at'].notna() & parsed.isna()
print("Same rows - raw:", int(raw_f['converted_at'].notna().sum()), "| parsed:", int(parsed.notna().
    sum()))
print(raw_f.loc[lost, 'converted_at'].astype(str).head(10))
```

WHY IT WORKS

- A before/after comparison only means something when both sides went through the same filters.
- Printing the failing values shows the cause instead of guessing it.

Lesson: Compare like with like: apply the same filters to both sides of a check.

20261111-07 All-time deposits for every client in the leads file

pandas · **How-to** · set of IDs, groupby, map

PROBLEM

Needed all-time deposits for every client in df, not only for the month's leads.

CAUSE

leads_id came from the filtered October leads, so clients with older leads had no figure.

FIX

```
df['zynixx_client_id'] = up(df['zynixx_client_id']).where(df['zynixx_client_id'].notna())
df_ids = set(df['zynixx_client_id'].dropna())                    # every client in the file
leads_led = led[led['ClientID'].isin(df_ids)]
all_dep_by_client = leads_led[leads_led['LedgerType'].eq('DEPOSIT')].groupby('ClientID')['Credit'].sum(
    )
df['All_Time_Deposit'] = df['zynixx_client_id'].map(all_dep_by_client).fillna(0)
```

WHY IT WORKS

- .where(notna) keeps blank IDs blank instead of turning them into the text 'NAN'.
- A client with several rows in df carries the full total on each row: deduplicate per client before summing per source.

Lesson: Build the ID list from the widest table the question needs, then filter later.

20261111-08 Filtering leads by lead date missed conversions of earlier leads

pandas · **Wrong result** · cohort vs activity, event date

PROBLEM

The October report kept only leads created in October, so a September lead that converted or deposited in October never counted.

CAUSE

One filter was answering two questions. New leads count by lead date; conversions, first deposits and deposits count by the date they happened, whatever month the lead came in.

FIX

```
clients = (df.dropna(subset=['zynixx_client_id'])
             .sort_values(['converted_at', 'lead_date'], na_position='last')      # converted lead
    first
             .drop_duplicates('zynixx_client_id'))                                 # one source per
    client

new = df[in_month(df['lead_date'])]                  # view 1: new leads, by lead date
conv = clients[in_month(clients['converted_at'])]    # view 2: events, by their own date
ftd = clients[in_month(clients['ftd_date'])]
```

WHY IT WORKS

- Every client needs one source: the lead that converted, else the earliest.
- converted_from_earlier_leads shows exactly what the old filter was missing.
- Which lead gets the credit (converting lead or last touch) is a rule to confirm with the team.

Lesson: Count each column by its own date, and write that date in the report's notes.

20261111-09 KeyError: 'converted_in_month' not found in columns (astype)

pandas · **Error** · stale cell, astype

PROBLEM

KeyError: "Only a column name can be used for the key in a dtype mappings argument. 'converted_in_month' not found in columns."

CAUSE

The new column was added to the counts list, but the cell that builds activity hadn't been run again, so the table in memory was the old one.

FIX

```
activity = pd.concat([..., df[in_month(df['converted_at'])].groupby('lead_source_1').size()
                                .rename('converted_in_month'), ...], axis=1)
print(activity.columns.tolist())        # the new name must be in this list
# then: Restart and Run All
```

WHY IT WORKS

- A notebook keeps variables from cells run earlier; editing a cell changes nothing until it runs.
- astype({...}) refuses names that aren't columns, so the error names the missing one.

Lesson: After editing a cell that others depend on, Restart and Run All.

20261111-10 One source under three labels: Meta-IMR, (META-IMR), Meta 1yt IMR

pandas · **Wrong result** · labels, replace, value_counts

PROBLEM

The (META-IMR) row showed 0 conversions and 0 FTD but 2,000 all-time deposit, against an expected 4 conversions.

CAUSE

Three spellings (1,587 / 306 / 1 leads) made three rows. The October conversions sat under Meta-IMR; the rest were November. FTD 0 with all-time 2,000 was right: that client deposited outside October.

FIX

```
imr = df[df['lead_source_1'].astype(str).str.contains('IMR', case=False, na=False)]
print(imr['lead_source_1'].value_counts(dropna=False))           # every spelling
df['lead_source_1'] = df['lead_source_1'].replace({'(META-IMR)': 'Meta-IMR', 'Meta 1yt IMR': 'Meta-
    IMR'})
```

WHY IT WORKS

- groupby treats every spelling as a different group.
- A replace dictionary merges only the labels named, so no other source changes.
- Of 6 converted IMR leads, 2 converted in October, 3 in November, 1 in September.

Lesson: Before grouping by a label, list its spellings with value_counts.

20261111-11 Mixed date formats: format='ISO8601' instead of errors='coerce'

pandas · **How-to** · to_datetime, ISO8601

PROBLEM

Some converted_at values had milliseconds and some didn't.

CAUSE

pandas guesses one format from the first value; with errors='coerce', anything that doesn't fit becomes blank without a warning.

FIX

```
df['lead_date'] = pd.to_datetime(df['lead_date'], format='ISO8601')
df['converted_at'] = pd.to_datetime(df['converted_at'], format='ISO8601')
```

WHY IT WORKS

- ISO8601 reads 2026-10-05 10:00:00 and 2026-10-05 10:00:00.537 alike.
- No errors='coerce': a truly broken date raises instead of silently removing a conversion.

Lesson: Never let errors='coerce' decide silently: count what it blanked, or don't use it.

20261111-12 The same report for any month: lead_summary(month)

pandas · **How-to** · function, MonthBegin, pd.concat dict

PROBLEM

Needed November alongside October with identical rules.

CAUSE

Everything that depends on the month moved into one function; facts that don't (first deposit, traded, all-time) are worked out once outside it.

FIX

```
MONTHS = ['2026-10', '2026-11']

def lead_summary(month):
    start = pd.Timestamp(month)
    end = start + pd.offsets.MonthBegin(1)
    in_month = lambda s: (s >= start) & (s < end)
    ...                                                  # month_deposit, trade_volume, views 1 and 2
    return out

summaries = {m: lead_summary(m) for m in MONTHS}
combined = pd.concat(summaries, names=['month']).reset_index(level=0)
```

WHY IT WORKS

- Column names became month-neutral (month_deposit), since they hold any month's figures.
- pd.concat on a dict uses the keys as a new 'month' column.
- A print line per month checks deposits against the ledger.

Lesson: When a report repeats per period, make the period a parameter.

20261111-13 PermissionError: [Errno 13] Permission denied writing the Excel file

Python · **Error** · PermissionError, open file, try/except

PROBLEM

PermissionError: [Errno 13] Permission denied: 'C:\Users\Administrator\Downloads\Lead_Source_Summary_2026-10.xlsx'

CAUSE

The file from an earlier run was open in Excel, and Windows locks open files.

FIX

```
try:
    export_lead_summary(s, path, pd.Timestamp(month), ledger_up_to=as_of)
except PermissionError:                                  # the file is open in Excel
    path = rf"{folder}\Lead_Source_Summary_{month}_{pd.Timestamp.now():%H%M%S}.xlsx"
    export_lead_summary(s, path, pd.Timestamp(month), ledger_up_to=as_of)
```

WHY IT WORKS

- Close the file (check minimised Excel windows) and run again; or save a timed copy automatically.
- Catching only PermissionError keeps every other error visible.

Lesson: Close output files in Excel before a run writes them.

20261111-14 Export: one column list drives the bands, month-to-date title, a sheet per month

Python · **How-to** · openpyxl, COLUMNS spec, ExcelWriter

PROBLEM

The export module had to take the new columns, any month, and several months in one file.

CAUSE

Bands, widths and formats were positional; adding a column meant editing several places.

FIX

```
COLUMNS = [("lead_source_1", "Lead Source", "text", ""),
           ("converted_in_month", "Converted", "int", "CONVERSIONS"),
           ("month_deposit", "{mon} Deposit", "money", "DEPOSITS"), ...]   # (column, header, kind,
    band)

export_lead_summaries(summaries, path, ledger_up_to=as_of)   # sheets 'Oct 2026', 'Nov 2026'
# title adds "(month to date)" when the ledger ends before the month does
```

WHY IT WORKS

- Bands are worked out from runs of the same band name, widths from the kind.
- A column missing from a summary is skipped instead of crashing.
- Recalculated: zero formula errors; every total matched pandas in both sheets.

Lesson: Describe a layout as data (one list), and let the code draw it.

20261111-15 TypeError: unexpected keyword argument 'temperature' (code from an older tutorial)

Python · **Error** · anthropic package, temperature, prefill

PROBLEM

Older tutorials call client.messages.create(..., temperature=0.2). With the anthropic package (version 2.8.0) the call fails before anything is sent.

CAUSE

Current Claude models don't accept sampling settings such as temperature (the API returns a 400 error), and the current Python package no longer takes the argument. Prefilling the start of the answer is refused on current models too.

FIX

```
client.messages.create(model=MODEL, max_tokens=10, temperature=0.2, messages=[...])
# TypeError: Messages.create() got an unexpected keyword argument 'temperature'

# steer with instructions instead
SYSTEM = "Reply with exactly one of: Interested, Not interested, Callback."
```

WHY IT WORKS

- Checked against the official Messages API documentation and the installed package.
- Instructions in the prompt control length and format more clearly anyway.
- When a tutorial's code fails, check its date before debugging your own code.

Lesson: Check a tutorial's date before trusting its arguments.

Thursday, 12 November 2026

Focus	The Pull Back call list by trade day (IST calendar), notebook errors after a kernel restart, a cross-check workbook behind the lead report, and Day 1 of building a language model from scratch.
Outcome	9 entries: 4 errors, 2 wrong results and 3 how-tos. The lead report now carries its own Clients, Leads and Check sheets; the first counting language model reaches a validation loss of 2.239.
Review	Repeated: setup lost on a restart (sys.path, helpers) and an old file shadowing a new one: keep setup in the first cell and check files exist. Open: the SLNo. check on repeated ledger rows (20261111-05) and the Thailand decisions. Next: the Django B0 self-check, then Build-an-LLM Day 2.
Cards	`20261112-01` to `20261112-09`

20261112-01 INTERVAL '3 days' is 72 hours, not three calendar days

SQL · **Wrong result** · AT TIME ZONE, date difference, CASE

PROBLEM

The Pull Back call list had to show only clients whose last trade was 2 or 3 days ago, tagged Second day / Third day, with today's and yesterday's traders left out.

CAUSE

last_txn_date >= CURRENT_TIMESTAMP - INTERVAL '3 days' counts back 72 hours: at 2 pm it starts at 2 pm three days earlier, so morning trades on day 3 were dropped.

FIX

```
WITH base AS (
    SELECT ctp.zynixx_client_id,
           (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date
             - (ctp.last_txn_date AT TIME ZONE 'Asia/Kolkata')::date AS days_since_last_txn
           -- other columns, joins and filters as before
    FROM customer_trading_profiles ctp
    WHERE ctp.last_txn_date >= CURRENT_TIMESTAMP - INTERVAL '5 days'      -- quick first cut only
)
SELECT *, CASE days_since_last_txn WHEN 2 THEN 'Second day' WHEN 3 THEN 'Third day' END AS trade_day
FROM base
WHERE days_since_last_txn IN (2, 3)
```

WHY IT WORKS

- Subtracting two IST dates gives whole calendar days: today 0, yesterday 1.
- One number drives both the filter and the label, so they can't disagree.
- The 5-day interval only narrows the rows; the calendar rule decides.

Lesson: Rolling hours and calendar days answer different questions: say which one the team means.

20261112-02 TypeError: Boolean array expected for the condition, not object

pandas · **Error** · where, brackets, KeyError

PROBLEM

up(df['zynixx_client_id']).where(df['zynixx_client_id']).notna() failed, and the next line parsed df['lead_Date'] into converted_at.

CAUSE

.notna() sat outside the bracket, so where() got the ID column (text) instead of a True/False condition. The next line had a capital D (KeyError) and the wrong column: it would have copied the lead date into converted_at.

FIX

```
df['zynixx_client_id'] = up(df['zynixx_client_id']).where(df['zynixx_client_id'].notna())
df['converted_at'] = pd.to_datetime(df['converted_at'], format='ISO8601')

print(df[['zynixx_client_id', 'lead_date', 'converted_at']].head())      # IDs, not True/False
```

WHY IT WORKS

- where(condition) keeps values where the condition is True; the condition must be True/False.
- Even if it had run, .notna() outside would have turned every ID into True or False.
- Column names are case-sensitive: lead_Date is not lead_date.

Lesson: Read a chained line bracket by bracket: what goes into each call?

20261112-03 ModuleNotFoundError: No module named 'ledger_loader' after a kernel restart

Python · **Error** · sys.path, restart, first cell

PROBLEM

After restarting the kernel, from ledger_loader import get_final_ledger failed.

CAUSE

A restart wipes everything, including the sys.path entry for the Office folder, so Python stopped looking there. The helpers up and KEY were gone too.

FIX

```
import sys

OFFICE = r"C:/Users/Administrator/Videos/Office"
if OFFICE not in sys.path:
    sys.path.insert(0, OFFICE)
from ledger_loader import get_final_ledger
up = lambda s: s.astype(str).str.strip().str.upper()

# once, for every notebook:   setx PYTHONPATH "C:\Users\Administrator\Videos\Office"
```

WHY IT WORKS

- Python only searches its standard folders, the notebook's own folder and sys.path.
- With setup in cell 1, a restart is fixed by running from the top.
- PYTHONPATH, or keeping notebooks in the Office folder, removes the need for the line.

Lesson: Put imports, paths and helpers in the first cell.

20261112-04 A sys.path entry missing its drive letter

Python · **Error** · Windows paths, os.path.isdir

PROBLEM

The first cell had sys.path.insert(0, r":/Users/Administrator/Videos/Office"), and the import still failed.

CAUSE

Without the C, Windows can't tell which drive is meant, so Python searched a folder that doesn't exist, without any warning.

FIX

```
sys.path.insert(0, r"C:/Users/Administrator/Videos/Office")

import os
print(os.path.isdir(r"C:/Users/Administrator/Videos/Office"))     # True: the folder exists
```

WHY IT WORKS

- sys.path accepts any text; it never checks that the folder is real.
- One isdir() line turns a silent typo into an obvious False.

Lesson: Check that a path exists before relying on it.

20261112-05 Clients, Leads and Check sheets behind the lead report

Excel · **How-to** · COUNTIFS, SUMIFS, INDEX/MATCH

PROBLEM

Needed to cross-check the monthly summaries against the rows behind them, inside Excel.

CAUSE

A summary on its own can't show where its numbers came from; the detail rows and a recount can.

FIX

```
export_lead_workbook(summaries, path, clients_detail, leads_detail, ledger_up_to=led['CreatedAt'].max(
    ))

Check, column D:  =IFERROR(INDEX('Oct 2026'!$L$6:$L$20, MATCH($B5, 'Oct 2026'!$A$6:$A$20, 0)), 0)
Check, column E:  =SUMIFS(Clients!$I$2:$I$300, Clients!$B$2:$B$300, $B5)
Check, column G:  =IF(ABS(D5-E5)<0.01, "OK", "CHECK")
```

WHY IT WORKS

- Clients has one row per client with each month's money; Leads has one row per lead.
- Tested: 80 checks OK, and a figure changed on purpose was flagged with its difference of 500.
- Filtering Leads or Clients by source shows exactly which rows make up a number.

Lesson: Ship the rows behind a summary, with a check that proves they agree.

20261112-06 ImportError: cannot import name 'export_lead_workbook'

Python · **Error** · old file, Downloads copy, reload

PROBLEM

from export_lead_summary import export_lead_workbook failed after downloading the new module.

CAUSE

Either the Office folder still held the old file (the download landed in Downloads, possibly renamed 'export_lead_summary (1).py'), or the notebook had imported the old version before the file was replaced.

FIX

```
office = r"C:/Users/Administrator/Videos/Office/export_lead_summary.py"
print("new version:", "def export_lead_workbook" in open(office, encoding="utf-8").read())

import importlib
import export_lead_summary
importlib.reload(export_lead_summary)              # or restart the kernel
```

WHY IT WORKS

- Python imports a module once per kernel; replacing the file changes nothing until a restart or reload.
- Browsers rename a second download, so the new code can sit beside the old under another name.

Lesson: After replacing a module, check the file Python actually loads, then restart.

20261112-07 Why the clients table keeps leads with no converted_at

pandas · **How-to** · NaT, ledger vs CRM

PROBLEM

Asked why the clients table doesn't drop rows whose converted_at is blank.

CAUSE

clients is every client with an ID, one source each, and it feeds FTD, deposits and trade volume, which come from the ledger. Dropping blank converted_at would remove real depositors.

FIX

```
no_conv = clients['converted_at'].isna()
print("Clients with no converted_at:", int(no_conv.sum()))
print("...of which deposited:", int((no_conv & clients['alltime_deposit'].gt(0)).sum()))

converted_clients = clients[clients['converted_at'].notna()]     # a separate table, if needed
```

WHY IT WORKS

- The sort puts converted leads first; blank dates only lose ties, they aren't removed.
- A blank date (NaT) is never inside a month, so conversion counts ignore it without a filter.

Lesson: Before dropping blanks, ask which numbers those rows feed.

20261112-08 A language model by counting: bigram and trigram

NumPy · **How-to** · tokens, probabilities, loss

PROBLEM

Day 1 of building an LLM from scratch: learn next-character probabilities from 32,033 names, and measure them.

CAUSE

Every LLM predicts the next token; the simplest version counts which token follows which.

FIX

```
N = np.zeros((27, 27), dtype=np.int64)             # row = this character, column = next
for w in train:
    padded = "." + w + "."
    for a, b in zip(padded, padded[1:]):
        N[stoi[a], stoi[b]] += 1
P = (N + 1) / (N + 1).sum(axis=1, keepdims=True)    # each row adds up to 1
# validation loss: untrained 3.296 -> bigram 2.451 -> trigram 2.239
```

WHY IT WORKS

- keepdims keeps the row totals as a column, so each row is divided by its own total.
- Loss = the average -log(probability of what really came next); lower is better.
- Each extra character of context multiplies the table by 27, which is why neural networks come next.

Lesson: Measure every model with one number, on data it hasn't seen.

20261112-09 +1 smoothing drowned a tiny dataset

NumPy · **Wrong result** · smoothing, data size

PROBLEM

A bigram model trained on 14 UAE place names wrote noise such as ' wn fwttikm ket msbu'.

CAUSE

14 names give only 122 pairs for a 441-cell table; adding 1 to every cell added 441 imaginary counts to 122 real ones.

FIX

```
k = 0.01                                           # small data: smooth lightly
PP = (NP + k) / (NP + k).sum(axis=1, keepdims=True)
# k = 1    -> ' wn fwttikm ket msbu', 'jhahqfalhqefsjjb'
# k = 0.01 -> 'ruma', 'jan', 'attaishaima'
```

WHY IT WORKS

- On the 32,033 names, k = 0.1 and k = 1 give the same validation loss (2.4514); k = 10 is worse.
- How much smoothing is right depends on how much data each row has.

Lesson: A setting that suits big data can swamp small data: test it on both.

Template: day page

Copy this page for each new day. Fill Focus in the morning and Outcome before you log off.

Date	
Focus	
Outcome	
Review	Repeated mistakes, gaps, one focus for tomorrow
Cards	_____ to _____

Index lines to add tonight

ID	Tool	Type	Title	Tags

End-of-day check

- ☐ Every solved problem has a card
- ☐ Error messages pasted word for word
- ☐ Cards added to the Index
- ☐ Any lesson seen before? Add it to Patterns

Template: log card

One card per problem. Two cards fit on a page when the fix is short.

ID		Tool		Type	Error / Wrong result / How-to
Title					
Tags					
Problem					
Cause					
Fix					
Why it works					
Lesson					

ID		Tool		Type	Error / Wrong result / How-to
Title					
Tags					
Problem					
Cause					
Fix					
Why it works					
Lesson					